MINI BANK PROJECT

Practical 1

1)	MiniBank.java
CODE	

```
import java.util.Scanner;

public class MiniBank
{

   // Record to store bank information
   record BankInfo(String name, String branch) {}

   // Enum for menu options
   enum MenuOption
   {
      OPEN_ACCOUNT,
      DEPOSIT,
      WITHDRAW,
      TRANSFER,
      EXIT
   }

   @SuppressWarnings("ConvertToTryWithResources")
   public static void main(String[] args)
   {

      Scanner sc = new Scanner(System.in);

      // Create and display bank information
      BankInfo bankInfo = new BankInfo("MiniBank", "Kheroli Branch");

      System.out.println("================================");
      System.out.println("      " + bankInfo.name());
      System.out.println("      Branch: " + bankInfo.branch());
      System.out.println("================================");

      while (true)
      {

         // Display menu
         System.out.println("\n------ Main Menu ------");
         System.out.println("1. Open Account");
         System.out.println("2. Deposit");
         System.out.println("3. Withdraw");
         System.out.println("4. Transfer");
         System.out.println("5. Exit");

         System.out.print("Enter your choice: ");
```

```
            int choice = sc.nextInt();

            // Switch expression
            String message = switch (choice)

            {
                case 1 -> "Open Account - To be implemented in a later
lab.";
                case 2 -> "Deposit - To be implemented in a later lab.";
                case 3 -> "Withdraw - To be implemented in a later lab.";
                case 4 -> "Transfer - To be implemented in a later lab.";
                case 5 -> "Thank you for using MiniBank. Goodbye!";
                default -> "Invalid choice! Please try again.";
            };

            System.out.println(message);

            if (choice == 5)
            {
                break;
            }
        }

        sc.close();
    }
}
```

OUTPUT

```
PS E:\java\25aiml044> java MiniBank.java
================================
      MiniBank
      Branch: CHANGA Branch
================================

------ Main Menu ------
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 1
Open Account - To be implemented in a later lab.

------ Main Menu ------
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 2
Deposit - To be implemented in a later lab.

------ Main Menu ------
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 3
Withdraw - To be implemented in a later lab.

------ Main Menu ------
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 4
Transfer - To be implemented in a later lab.

------ Main Menu ------
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 5
Thank you for using MiniBank. Goodbye!
PS E:\java\25aiml044>
```

Practical 2

	Customer.java

CODE	public class Customer {
	  private String name;
	  private String email;
	  private String mobile;
	  private final String customerId;
	
	  private static long customerCounter = 100;
	
	  public Customer(String name, String email, String mobile) {
	  this.name = name;
	  this.email = email;
	  this.mobile = mobile;
	  this.customerId = generateCustomerId();
	  }
	
	  private static String generateCustomerId() {
	  customerCounter++;
	  return "CUST" + customerCounter;
	  }
	
	  public String getName() {
	  return name;
	  }
	
	  public String getEmail() {
	  return email;
	  }
	
	  public String getMobile() {
	  return mobile;
	  }
	
	  public String getCustomerId() {
	  return customerId;
	  }
	}

	Account.java
CODE	public class Account { private final String accountNumber; private String ownerName; private long balance; private boolean active; private static long accountCounter = 0;

```
    public Account(String ownerName, long openingBalance) {
        accountCounter++;
        this.accountNumber = String.format("AC%04d",
accountCounter);
        this.ownerName = ownerName;
        this.balance = openingBalance >= 0 ? openingBalance : 0;
        this.active = true;
    }

    public Account(String ownerName) {
        this(ownerName, 0);
    }

    public void deposit(long amount) {
        if (amount <= 0) {
            System.out.println("Deposit amount must be greater than
zero.");
            return;
        }

        balance += amount;
    }

    public boolean withdraw(long amount) {
        if (amount <= 0) {
            System.out.println("Withdrawal amount must be greater than
zero.");
            return false;
        }

        if (balance >= amount) {
            balance -= amount;
            return true;
        }

        return false;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    public long getBalance() {
        return balance;
    }
```

```
    public boolean isActive() {
        return active;
    }

    public static boolean transfer(Account from, Account to, long
amount) {
        if (from == null || to == null || amount <= 0) {
            return false;
        }

        if (from.withdraw(amount)) {
            to.deposit(amount);
            return true;
        }

        return false;
    }
}
```

MiniBank.java

CODE

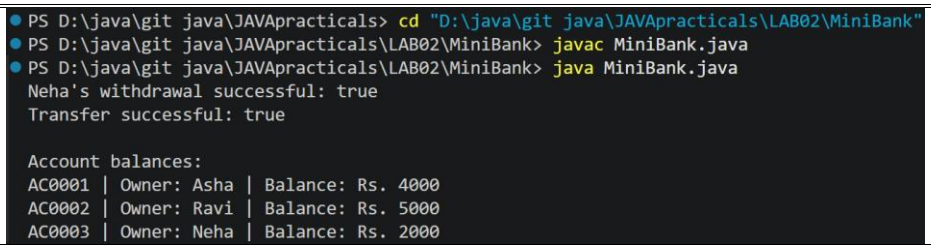
```
public class MiniBank {
    public static void main(String[] args) {
        Account[] accounts = {
            new Account("Asha", 5000),
            new Account("Ravi"),
            new Account("Neha", 2500)
        };

        accounts[0].deposit(1000);
        accounts[1].deposit(3000);

        boolean withdrawalSuccessful = accounts[2].withdraw(500);
        System.out.println("Neha's withdrawal successful: " +
withdrawalSuccessful);

        boolean transferSuccessful =
                Account.transfer(accounts[0], accounts[1], 2000);
        System.out.println("Transfer successful: " +
transferSuccessful);

        System.out.println("\nAccount balances:");
        for (Account account : accounts) {
            System.out.println(
                    account.getAccountNumber()
                    + " | Owner: " + account.getOwnerName()
                    + " | Balance: Rs. " + account.getBalance()
            );
        }
    }
}
```

OUTPUT	

Practical 3

	Customer.java
CODE	public class Customer implements Cloneable { private String name; private String email; private String mobile; private final String customerId;

```
    private Address address;

    private static long customerCounter = 100;

    public Customer(String name, String email, String mobile, Address
address) {
        this.name = name;
        this.email = email;
        this.mobile = mobile;
        this.address = address;
        this.customerId = generateCustomerId();
    }

    private static String generateCustomerId() {
        customerCounter++;
        return "CUST" + customerCounter;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getMobile() {
        return mobile;
    }

    public String getCustomerId() {
        return customerId;
    }

    public Address getAddress() {
        return address;
    }

    @Override
    public Customer clone() {
        try {
            return (Customer) super.clone();
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);
        }
    }

    public static class Address {
        private String line;
        private String city;
```

```
        private String pincode;

        public Address(String line, String city, String pincode) {
            this.line = line;
            this.city = city;
            this.pincode = pincode;
        }

        public String getLine() {
            return line;
        }

        public String getCity() {
            return city;
        }

        public String getPincode() {
            return pincode;
        }

        @Override
        public String toString() {
            return line + ", " + city + " - " + pincode;
        }
    }
}
```

Account.java

CODE

```
import java.util.Objects;

public class Account {
    private final String accountNumber;
    private String ownerName;
    private long balance;
    private boolean active;

    private static int accountCounter = 0;

    public Account(String ownerName, long openingBalance) {
        accountCounter++;
        this.accountNumber = String.format("AC%04d",
accountCounter);
        this.ownerName = ownerName;
        this.balance = Math.max(openingBalance, 0);
        this.active = true;
    }

    public Account(String ownerName) {
        this(ownerName, 0);
    }
```

```
    public boolean deposit(long amount) {
        if (amount <= 0) {
            return false;
        }

        balance += amount;
        return true;
    }

    public boolean withdraw(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }

        balance -= amount;
        return true;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    public long getBalance() {
        return balance;
    }

    public boolean isActive() {
        return active;
    }

    @Override
    public String toString() {
        return "Account Number: " + accountNumber
                + " | Owner: " + ownerName
                + " | Balance: Rs. " + balance
                + " | Active: " + active;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }

        if (!(o instanceof Account)) {
```

```
            return false;
        }

        Account account = (Account) o;
        return accountNumber.equals(account.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }

    public String getStatement() {
        return "----- Account Statement -----\n"
                + "Account Number: " + accountNumber + "\n"
                + "Owner: " + ownerName + "\n"
                + "Balance: Rs. " + balance + "\n"
                + "Active: " + active + "\n"
                + "----------------------------";
    }
}
```

MiniBank.java

CODE

```
import java.util.Scanner;

public class MiniBank {
    enum MenuOption {
        OPEN_ACCOUNT,
        DEPOSIT,
        WITHDRAW,
        TRANSFER,
        EXIT
    }

    record BankInfo(String name, String branch) {
    }

    public static void main(String[] args) {
        BankInfo bankInfo = new BankInfo("MiniBank", "Kheroli Branch");
        Scanner s = new Scanner(System.in);

        Account[] accounts = {
            new Account("Bhavya", 5000),
            new Account("Japan", 10000),
            new Account("Shivam")
        };

        Customer.Address address =
                new Customer.Address("Main Road", "Kheroli", "123456");
```

```
        Customer customer = new Customer(
                "Bhavya",
                "bhavya@example.com",
                "9876543210",
                address
        );

        Customer copiedCustomer = customer.clone();

        System.out.println("================================");
        System.out.println("      " + bankInfo.name());
        System.out.println("      Branch: " + bankInfo.branch());
        System.out.println("================================");

        // Print accounts using toString()
        System.out.println("\nAccounts:");
        for (Account account : accounts) {
            System.out.println(account.toString());
        }

        // Compare two accounts using equals()
        System.out.println("\nAre the first two accounts equal? "
                + accounts[0].equals(accounts[1]));

        // Check an object's type using instanceof
        Object object = accounts[0];
        if (object instanceof Account) {
            System.out.println("The object is an Account.");
        }

        System.out.println("\nCustomer: " + copiedCustomer.getName());
        System.out.println("Customer ID: " +
copiedCustomer.getCustomerId());
        System.out.println("Address: " + copiedCustomer.getAddress());

        // Print the formatted account statement
        System.out.println("\n" + accounts[0].getStatement());

        boolean run = true;

        while (run) {
            System.out.println("\n------ Main Menu ------");
            System.out.println("1) Open Account");
            System.out.println("2) Deposit");
            System.out.println("3) Withdraw");
            System.out.println("4) Transfer");
            System.out.println("5) Exit");
            System.out.print("Enter your choice: ");

            int choice = s.nextInt();
```

```
            MenuOption option;

            switch (choice) {
                case 1 -> option = MenuOption.OPEN_ACCOUNT;
                case 2 -> option = MenuOption.DEPOSIT;
                case 3 -> option = MenuOption.WITHDRAW;
                case 4 -> option = MenuOption.TRANSFER;
                case 5 -> option = MenuOption.EXIT;
                default -> option = null;
            }

            if (option == null) {
                System.out.println("Invalid choice. Please try again.");
                continue;
            }

            switch (option) {
                case OPEN_ACCOUNT -> {
                    System.out.println("\nAccounts:");
                    for (Account account : accounts) {
                        System.out.println(account);
                    }
                }

                case DEPOSIT -> {
                    int index = selectAccount(s, accounts);
                    if (index == -1) {
                        break;
                    }

                    System.out.print("Enter amount: ");
                    long amount = s.nextLong();

                    if (accounts[index].deposit(amount)) {
                        System.out.println("Deposit successful.");
                    } else {
                        System.out.println("Amount must be greater than
zero.");
                    }

                    System.out.println(accounts[index]);
                }

                case WITHDRAW -> {
                    int index = selectAccount(s, accounts);
                    if (index == -1) {
                        break;
                    }

                    System.out.print("Enter amount: ");
```

```
                long amount = s.nextLong();

                if (accounts[index].withdraw(amount)) {
                    System.out.println("Withdrawal successful.");
                } else {
                    System.out.println(
                            "Withdrawal failed. Check the amount and
balance."
                    );
                }

                System.out.println(accounts[index]);
            }

            case TRANSFER ->
                System.out.println("Transfer feature will be implemented
later.");

            case EXIT -> {
                System.out.println("Thank you for using MiniBank.
Goodbye!");
                run = false;
            }
        }
    }

    s.close();
  }

  private static int selectAccount(Scanner s, Account[] accounts) {
    System.out.println("Select account:");

    for (int i = 0; i < accounts.length; i++) {
      System.out.println((i + 1) + ". " + accounts[i].getOwnerName());
    }

    int choice = s.nextInt();

    if (choice < 1 || choice > accounts.length) {
      System.out.println("Invalid account.");
      return -1;
    }

    return choice - 1;
  }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals> cd "D:\java\git java\JAVApracticals\LAB03\MiniBank"
PS D:\java\git java\JAVApracticals\LAB03\MiniBank> javac MiniBank.java
PS D:\java\git java\JAVApracticals\LAB03\MiniBank> java MiniBank.java
 ================================
       MiniBank
       Branch: Kheroli Branch
 ================================

 Accounts:
 Account Number: AC0001 | Owner: Bhavya | Balance: Rs. 5000 | Active: true
 Account Number: AC0002 | Owner: Japan | Balance: Rs. 10000 | Active: true
 Account Number: AC0003 | Owner: Shivam | Balance: Rs. 0 | Active: true

 Are the first two accounts equal? false
 The object is an Account.

 Customer: Bhavya
 Customer ID: CUST101
 Address: Main Road, Kheroli - 123456

 ----- Account Statement -----
 Account Number: AC0001
 Owner: Bhavya
 Balance: Rs. 5000
 Active: true
 ----------------------------

 ------ Main Menu ------
 1) Open Account
 2) Deposit
 3) Withdraw
 4) Transfer
 5) Exit
 Enter your choice: 1
PS D:\java\git java\JAVApracticals\LAB03\MiniBank> java MiniBank.java
Accounts:
Account Number: AC0001 | Owner: Bhavya | Balance: Rs. 5000 | Active: true
Account Number: AC0002 | Owner: Japan | Balance: Rs. 10000 | Active: true
Account Number: AC0003 | Owner: Shivam | Balance: Rs. 0 | Active: true

------ Main Menu ------
1) Open Account
2) Deposit
3) Withdraw
4) Transfer
5) Exit
Enter your choice: 2
Select account:
1. Bhavya
2. Japan
3. Shivam
1
Enter amount: 2000
Deposit successful.
Account Number: AC0001 | Owner: Bhavya | Balance: Rs. 7000 | Active: true

------ Main Menu ------
1) Open Account
2) Deposit
3) Withdraw
4) Transfer
5) Exit
Enter your choice: 4
Transfer feature will be implemented later.

------ Main Menu ------
1) Open Account
2) Deposit
3) Withdraw
```

```
------ Main Menu ------
1) Open Account
2) Deposit
3) Withdraw
4) Transfer
5) Exit
Enter your choice: 4
Transfer feature will be implemented later.

------ Main Menu ------
1) Open Account
2) Deposit
3) Withdraw
4) Transfer
5) Exit
Enter your choice: 5
Thank you for using MiniBank. Goodbye!
PS D:\java\git java\JAVApracticals\LAB03\MiniBank>
```

Practical 4

	Customer.java

CODE	public class Customer implements Cloneable {
	  private String name;
	  private String email;
	  private String mobile;
	  private final String customerId;
	  private Address address;
	
	  private static long customerCounter = 100;
	
	  public Customer(String name, String email, String mobile, Address address) {
	  this.name = name;
	  this.email = email;
	  this.mobile = mobile;
	  this.address = address;
	  this.customerId = generateCustomerId();
	  }
	
	  private static String generateCustomerId() {
	  customerCounter++;
	  return "CUST" + customerCounter;
	  }
	
	  public String getName() {
	  return name;
	  }
	
	  public String getEmail() {
	  return email;
	  }
	
	  public String getMobile() {
	  return mobile;
	  }
	
	  public String getCustomerId() {
	  return customerId;
	  }
	
	  public Address getAddress() {
	  return address;
	  }
	
	  @Override
	  public Customer clone() {
	  try {

```
            return (Customer) super.clone();
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);
        }
    }

    public static class Address {
        private String line;
        private String city;
        private String pincode;

        public Address(String line, String city, String pincode) {
            this.line = line;
            this.city = city;
            this.pincode = pincode;
        }

        public String getLine() {
            return line;
        }

        public String getCity() {
            return city;
        }

        public String getPincode() {
            return pincode;
        }

        @Override
        public String toString() {
            return line + ", " + city + " - " + pincode;
        }
    }
}
```

	Account.java
CODE	

```
import java.util.Objects;

public class Account {
    private final String accountNumber;
    private String ownerName;
    private long balance;
    private boolean active;

    private static int accountCounter = 0;

    public Account(String ownerName, long openingBalance) {
        accountCounter++;
```

```
        this.accountNumber = String.format("AC%04d",
accountCounter);
        this.ownerName = ownerName;
        this.balance = Math.max(openingBalance, 0);
        this.active = true;
    }

    public Account(String ownerName) {
        this(ownerName, 0);
    }

    public boolean deposit(long amount) {
        if (amount <= 0) {
            return false;
        }

        balance += amount;
        return true;
    }

    public boolean withdraw(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }

        balance -= amount;
        return true;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    public long getBalance() {
        return balance;
    }

    public boolean isActive() {
        return active;
    }

    @Override
    public String toString() {
        return "Account Number: " + accountNumber
                + " | Owner: " + ownerName
                + " | Balance: Rs. " + balance
```

```
                + " | Active: " + active;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }

        if (!(o instanceof Account)) {
            return false;
        }

        Account account = (Account) o;
        return accountNumber.equals(account.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}
```

MiniBank.java

CODE

```
import java.util.regex.Pattern;

public class MiniBank {

    public static void main(String[] args) {
        System.out.println("Mobile tests:");
        System.out.println("Valid:   " +
Validator.isValidMobile("9876543210"));
        System.out.println("Invalid: " +
Validator.isValidMobile("12345"));

        System.out.println("\nEmail tests:");
        System.out.println("Valid:   " +
Validator.isValidEmail("user@example.com"));
        System.out.println("Invalid: " +
Validator.isValidEmail("user@"));

        System.out.println("\nPAN tests:");
        System.out.println("Valid:   " +
Validator.isValidPan("ABCDE1234F"));
        System.out.println("Invalid: " +
Validator.isValidPan("ABC123"));

        System.out.println("\nIFSC tests:");
        System.out.println("Valid:   " +
Validator.isValidIfsc("SBIN0001234"));
```

```
        System.out.println("Invalid: " + Validator.isValidIfsc("123"));

        System.out.println("\nAmount tests:");
        System.out.println("Valid:   " + Validator.isValidAmount("500"));
        System.out.println("Invalid: " + Validator.isValidAmount("-10"));

        String line = "DEPOSIT AC0001 500";

        try {
            Command command = CommandParser.parse(line);

            System.out.println("\nParsed command:");
            System.out.println("Type: " + command.type());
            System.out.println("Account number: " +
command.accountNumber());
            System.out.println("Amount: " + command.amount());

            Account account = new Account("Bhavya", 5000);
            System.out.println("\n" +
StatementFormatter.buildStatement(account));
        } catch (IllegalArgumentException e) {
            System.out.println("Command error: " + e.getMessage());
        }
    }
}

class Validator {
    private static final Pattern MOBILE_PATTERN =
            Pattern.compile("^[6-9][0-9]{9}$");

    private static final Pattern EMAIL_PATTERN =
            Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-
z]{2,}$");

    private static final Pattern PAN_PATTERN =
            Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");

    private static final Pattern IFSC_PATTERN =
            Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");

    private static final Pattern AMOUNT_PATTERN =
            Pattern.compile("^[1-9][0-9]*$");

    private Validator() {
    }

    public static boolean isValidMobile(String mobile) {
        return mobile != null &&
MOBILE_PATTERN.matcher(mobile).matches();
    }
```

```
    public static boolean isValidEmail(String email) {
        return email != null &&
EMAIL_PATTERN.matcher(email).matches();
    }

    public static boolean isValidPan(String pan) {
        return pan != null && PAN_PATTERN.matcher(pan).matches();
    }

    public static boolean isValidIfsc(String ifsc) {
        return ifsc != null && IFSC_PATTERN.matcher(ifsc).matches();
    }

    public static boolean isValidAmount(String amount) {
        if (amount == null ||
!AMOUNT_PATTERN.matcher(amount).matches()) {
            return false;
        }

        try {
            return Long.parseLong(amount) > 0;
        } catch (NumberFormatException e) {
            return false;
        }
    }
}

enum TransactionType {
    DEPOSIT,
    WITHDRAW,
    TRANSFER
}

record Command(TransactionType type, String accountNumber,
long amount) {
}

class CommandParser {
    private CommandParser() {
    }

    public static Command parse(String line) {
        if (line == null || line.isBlank()) {
            throw new IllegalArgumentException("Command cannot be
empty.");
        }

        String[] parts = line.trim().split("\\s+");
```

```
        if (parts.length != 3) {
            throw new IllegalArgumentException(
                    "Command must have three parts: TYPE
ACCOUNT_NUMBER AMOUNT."
            );
        }

        try {
            TransactionType type = TransactionType.valueOf(
                    parts[0].toUpperCase()
            );

            String accountNumber = parts[1];

            if (!accountNumber.matches("AC[0-9]{4}")) {
                throw new IllegalArgumentException("Invalid account
number.");
            }

            if (!Validator.isValidAmount(parts[2])) {
                throw new IllegalArgumentException(
                        "Amount must be a positive whole number."
                );
            }

            long amount = Long.parseLong(parts[2]);
            return new Command(type, accountNumber, amount);

        } catch (IllegalArgumentException e) {
            if (e instanceof IllegalArgumentException
                    && e.getMessage() != null
                    && (e.getMessage().startsWith("Invalid")
                    || e.getMessage().startsWith("Amount"))) {
                throw e;
            }

            throw new IllegalArgumentException(
                    "Unknown transaction type: " + parts[0]
            );
        }
    }
}

class StatementFormatter {
    private StatementFormatter() {
    }

    public static String buildStatement(Account account) {
        if (account == null) {
```

```
            throw new IllegalArgumentException("Account cannot be
null.");
        }

        StringBuilder statement = new StringBuilder();
        statement.append("------ Account Statement ------\n");
        statement.append("Account number: ")
                .append(account.getAccountNumber()).append("\n");
        statement.append("Owner: ")
                .append(account.getOwnerName()).append("\n");
        statement.append("Balance: Rs. ")
                .append(account.getBalance()).append("\n");
        statement.append("Active: ")
                .append(account.isActive()).append("\n");
        statement.append("-----------------------------");

        return statement.toString();
    }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals> cd "D:\java\git java\JAVApracticals\LAB04\MiniBank"
PS D:\java\git java\JAVApracticals\LAB04\MiniBank> javac MiniBank.java
PS D:\java\git java\JAVApracticals\LAB04\MiniBank> java MiniBank.java
Mobile tests:
Valid:   true
Invalid: false

Email tests:
Valid:   true
Invalid: false

PAN tests:
Valid:   true
Invalid: false

IFSC tests:
Valid:   true
Invalid: false

Amount tests:
Valid:   true
Invalid: false

Parsed command:
Type: DEPOSIT
Account number: AC0001
Amount: 500

------ Account Statement ------
Account number: AC0001
Owner: Bhavya
Balance: Rs. 5000
Active: true
-----------------------------
PS D:\java\git java\JAVApracticals\LAB04\MiniBank>
```

Practical 5

	Customer.java

CODE	public class Customer implements Cloneable {
	  private String name;
	  private String email;
	  private String mobile;
	  private final String customerId;
	  private Address address;
	
	  private static long customerCounter = 100;
	
	  public Customer(String name, String email, String mobile, Address address) {
	  this.name = name;
	  this.email = email;
	  this.mobile = mobile;
	  this.address = address;
	  this.customerId = generateCustomerId();
	  }
	
	  private static String generateCustomerId() {
	  customerCounter++;
	  return "CUST" + customerCounter;
	  }
	
	  public String getName() {
	  return name;

```
    }

    public String getEmail() {
        return email;
    }

    public String getMobile() {
        return mobile;
    }

    public String getCustomerId() {
        return customerId;
    }

    public Address getAddress() {
        return address;
    }

    @Override
    public Customer clone() {
        try {
            return (Customer) super.clone();
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);
        }
    }

    public static class Address {
        private String line;
        private String city;
        private String pincode;

        public Address(String line, String city, String pincode) {
            this.line = line;
            this.city = city;
            this.pincode = pincode;
        }

        public String getLine() {
            return line;
        }

        public String getCity() {
            return city;
        }

        public String getPincode() {
            return pincode;
        }
```

```
        @Override
        public String toString() {
            return line + ", " + city + " - " + pincode;
        }
    }
}
```

Account.java

CODE

```
import java.util.Objects;

public abstract class Account {
    private final String accountNumber;
    private String ownerName;
    private long balance;
    private boolean active;

    private static int accountCounter = 0;

    public Account(String ownerName, long openingBalance) {
        accountCounter++;
        this.accountNumber = String.format("AC%04d",
accountCounter);
        this.ownerName = ownerName;
        this.balance = openingBalance;
        this.active = true;
    }

    public Account(String ownerName) {
        this(ownerName, 0);
    }

    public abstract double interestRate();

    public abstract boolean canWithdraw(long amount);

    public boolean deposit(long amount) {
        if (amount <= 0) {
            return false;
        }

        balance += amount;
        return true;
    }

    public boolean withdraw(long amount) {
        if (amount <= 0 || !canWithdraw(amount)) {
            return false;
        }

        balance -= amount;
```

```
        return true;
    }

    public double monthlyInterest() {
        return balance * (interestRate() / 100.0) / 12.0;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    public long getBalance() {
        return balance;
    }

    public boolean isActive() {
        return active;
    }

    @Override
    public String toString() {
        return "Account Number: " + accountNumber
                + " | Owner: " + ownerName
                + " | Balance: Rs. " + balance
                + " | Active: " + active;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }

        if (!(o instanceof Account)) {
            return false;
        }

        Account account = (Account) o;
        return accountNumber.equals(account.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}
```

```
class SavingsAccount extends Account {
    private long minBalance;

    public SavingsAccount(String ownerName, long openingBalance,
long minBalance) {
        super(ownerName, openingBalance);
        this.minBalance = minBalance;
    }

    @Override
    public double interestRate() {
        return 4.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return amount > 0 && getBalance() - amount >= minBalance;
    }

    public long getMinBalance() {
        return minBalance;
    }
}

class CurrentAccount extends Account {
    private long overdraftLimit;

    public CurrentAccount(String ownerName, long openingBalance,
long overdraftLimit) {
        super(ownerName, openingBalance);
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    public double interestRate() {
        return 0.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return amount > 0 && getBalance() - amount >= -overdraftLimit;
    }

    public long getOverdraftLimit() {
        return overdraftLimit;
    }
}

class FixedDepositAccount extends Account {
```

```
    public FixedDepositAccount(String ownerName, long
openingBalance) {
        super(ownerName, openingBalance);
    }

    @Override
    public double interestRate() {
        return 7.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return false;
    }
}
```

MiniBank.java

CODE

```
public class MiniBank {
    public static void main(String[] args) {
        Account[] accounts = {
            new SavingsAccount("Bhavya", 5000, 1000),
            new CurrentAccount("Japan", 10000, 2000),
            new FixedDepositAccount("Shivam", 20000)
        };

        System.out.println("MiniBank Account Details");
        System.out.println("========================");

        for (Account account : accounts) {
            System.out.println(account);
            System.out.println("Interest rate: " + account.interestRate() +
"%");
            System.out.printf("Estimated monthly interest: Rs. %.2f%n",
                    account.monthlyInterest());

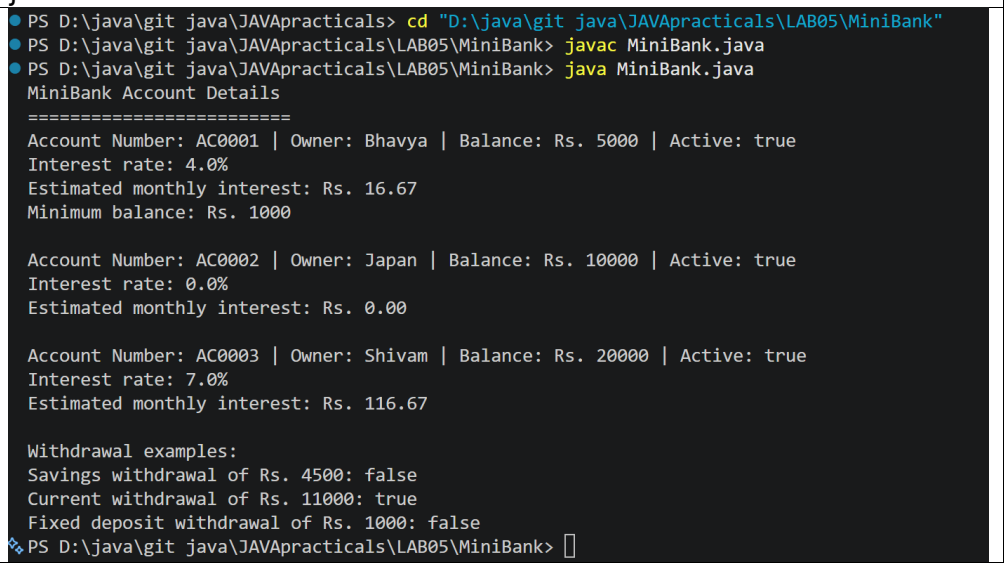
            if (account instanceof SavingsAccount savingsAccount) {
                System.out.println("Minimum balance: Rs. "
                        + savingsAccount.getMinBalance());
            }

            System.out.println();
        }

        // Demonstrate each account's withdrawal rule.
        System.out.println("Withdrawal examples:");
        System.out.println("Savings withdrawal of Rs. 4500: "
                + accounts[0].withdraw(4500));
        System.out.println("Current withdrawal of Rs. 11000: "
                + accounts[1].withdraw(11000));
        System.out.println("Fixed deposit withdrawal of Rs. 1000: "
```

```
                + accounts[2].withdraw(1000));
        }
}
```

OUTPUT



Practical 6

	MiniBank/model/Transactable.java

CODE	package model;
	
	public interface Transactable {
	  void deposit(long amount);
	
	  boolean withdraw(long amount);
	}

	MiniBank/model/InterestBearing.java

CODE	package model;
	
	public interface InterestBearing {
	  double interestRate();
	
	  long getBalance();
	
	  default double yearlyInterest() {
	  return getBalance() * interestRate() / 100.0;
	  }
	}

	MiniBank/model/WithdrawRule.java

CODE	package model;
	
	@FunctionalInterface
	public interface WithdrawRule {
	  boolean allow(Account account, long amount);
	}

	MiniBank/model/Premium.java

CODE	package model;
	
	public interface Premium {
	}

	MiniBank/model/Account.java
CODE	package model; import java.util.Objects; public abstract class Account implements Transactable, InterestBearing { private final String accountNumber; private final String ownerName; private long balance; private final boolean active;

```
    private static int accountCounter = 0;

    protected Account(String ownerName, long openingBalance) {
        accountCounter++;
        this.accountNumber = String.format("AC%04d",
accountCounter);
        this.ownerName = ownerName;
        this.balance = Math.max(openingBalance, 0);
        this.active = true;
    }

    public abstract boolean canWithdraw(long amount);

    @Override
    public void deposit(long amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    @Override
    public boolean withdraw(long amount) {
        if (amount <= 0 || !canWithdraw(amount)) {
            return false;
        }

        balance -= amount;
        return true;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    @Override
    public long getBalance() {
        return balance;
    }

    public boolean isActive() {
        return active;
    }

    @Override
    public String toString() {
        return "Account Number: " + accountNumber
```

```
                + " | Owner: " + ownerName
                + " | Balance: Rs. " + balance
                + " | Active: " + active;
    }

    @Override
    public boolean equals(Object object) {
        if (this == object) {
            return true;
        }

        if (!(object instanceof Account account)) {
            return false;
        }

        return accountNumber.equals(account.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}

class SavingsAccount extends Account implements Premium {
    private final long minBalance;

    public SavingsAccount(String ownerName, long openingBalance,
long minBalance) {
        super(ownerName, openingBalance);
        this.minBalance = minBalance;
    }

    @Override
    public double interestRate() {
        return 4.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return amount > 0 && getBalance() - amount >= minBalance;
    }

    public long getMinBalance() {
        return minBalance;
    }
}

class CurrentAccount extends Account {
    private final long overdraftLimit;
```

```
    public CurrentAccount(String ownerName, long openingBalance,
long overdraftLimit) {
        super(ownerName, openingBalance);
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    public double interestRate() {
        return 0.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return amount > 0 && getBalance() - amount >= -
overdraftLimit;
    }
}

class FixedDepositAccount extends Account {
    public FixedDepositAccount(String ownerName, long
openingBalance) {
        super(ownerName, openingBalance);
    }

    @Override
    public double interestRate() {
        return 7.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return false;
    }
}
```

MiniBank/model/Customer.java

CODE

```
package model;

public class Customer implements Cloneable {
    private String name;
    private String email;
    private String mobile;
    private final String customerId;
    private Address address;

    private static long customerCounter = 100;

    public Customer(String name, String email, String mobile,
Address address) {
        this.name = name;
```

```
        this.email = email;
        this.mobile = mobile;
        this.address = address;
        this.customerId = generateCustomerId();
    }

    private static String generateCustomerId() {
        customerCounter++;
        return "CUST" + customerCounter;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getMobile() {
        return mobile;
    }

    public String getCustomerId() {
        return customerId;
    }

    public Address getAddress() {
        return address;
    }

    @Override
    public Customer clone() {
        try {
            return (Customer) super.clone();
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);
        }
    }

    public static class Address {
        private final String line;
        private final String city;
        private final String pincode;

        public Address(String line, String city, String pincode) {
            this.line = line;
            this.city = city;
            this.pincode = pincode;
        }
```

```
    public String getLine() {
        return line;
    }

    public String getCity() {
        return city;
    }

    public String getPincode() {
        return pincode;
    }

    @Override
    public String toString() {
        return line + ", " + city + " - " + pincode;
    }
  }
}
```

MiniBank/util/Validator.java

CODE

```
package util;

import java.util.regex.Pattern;

public class Validator {
  private static final Pattern MOBILE_PATTERN =
      Pattern.compile("^[6-9][0-9]{9}$");

  private static final Pattern EMAIL_PATTERN =
      Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-
z]{2,}$");

  private static final Pattern PAN_PATTERN =
      Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");

  private static final Pattern IFSC_PATTERN =
      Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");

  private Validator() {
  }

  public static boolean isValidMobile(String mobile) {
    return mobile != null &&
MOBILE_PATTERN.matcher(mobile).matches();
  }

  public static boolean isValidEmail(String email) {
    return email != null &&
EMAIL_PATTERN.matcher(email).matches();
  }
```

```
    public static boolean isValidPan(String pan) {
        return pan != null && PAN_PATTERN.matcher(pan).matches();
    }

    public static boolean isValidIfsc(String ifsc) {
        return ifsc != null && IFSC_PATTERN.matcher(ifsc).matches();
    }
}
```

MiniBank/service/MiniBank.java

CODE

```
package service;

import model.Account;
import model.CurrentAccount;
import model.FixedDepositAccount;
import model.Premium;
import model.SavingsAccount;
import model.WithdrawRule;

import static util.Validator.isValidMobile;

public class MiniBank {
    public static void main(String[] args) {
        Account[] accounts = {
            new SavingsAccount("Bhavya", 5000, 1000),
            new CurrentAccount("Japan", 10000, 2000),
            new FixedDepositAccount("Shivam", 20000)
        };

        System.out.println("MiniBank Accounts");
        System.out.println("================");

        for (Account account : accounts) {
            System.out.println(account);
            System.out.println("Interest rate: " + account.interestRate() +
"%");
            System.out.println("Yearly interest: Rs. "
                    + account.yearlyInterest());

            if (account instanceof SavingsAccount savings) {
                System.out.println("Minimum balance: Rs. "
                        + savings.getMinBalance());
            }
            System.out.println();
        }

        // WithdrawRule used with an anonymous class.
        WithdrawRule anonymousRule = new WithdrawRule() {
            @Override
            public boolean allow(Account account, long amount) {
```

```
                return account.canWithdraw(amount);
            }
        };

        // WithdrawRule used with a lambda expression.
        WithdrawRule lambdaRule =
                (account, amount) -> account.canWithdraw(amount);

        System.out.println("Anonymous rule allows withdrawal: "
                + anonymousRule.allow(accounts[0], 1000));
        System.out.println("Lambda rule allows withdrawal: "
                + lambdaRule.allow(accounts[0], 1000));

        System.out.println("Bhavya has Premium status: "
                + (accounts[0] instanceof Premium));

        // Static import lets us call the validator without Validator.
        System.out.println("Mobile number valid: "
                + isValidMobile("9876543210"));
    }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB06\MiniBank> javac -d out (Get-ChildItem -Recurse -Filte
PS D:\java\git java\JAVApracticals\LAB06\MiniBank> jar --create --file MiniBank.jar --main-cla
>> java -jar MiniBank.jar
MiniBank Accounts
================
Account Number: AC0001 | Owner: Bhavya | Balance: Rs. 5000 | Active: true
Interest rate: 4.0%
Yearly interest: Rs. 200.0
Minimum balance: Rs. 1000

Account Number: AC0002 | Owner: Japan | Balance: Rs. 10000 | Active: true
Interest rate: 0.0%
Yearly interest: Rs. 0.0

Account Number: AC0003 | Owner: Shivam | Balance: Rs. 20000 | Active: true
Interest rate: 7.0%
Yearly interest: Rs. 1400.0

Anonymous rule allows withdrawal: true
Lambda rule allows withdrawal: true
Bhavya has Premium status: true
Mobile number valid: true
PS D:\java\git java\JAVApracticals\LAB06\MiniBank>
```

Practical 7

	model/annotation/Id.java

CODE	package model.annotation;
	
	import java.lang.annotation.ElementType;
	import java.lang.annotation.Retention;
	import java.lang.annotation.RetentionPolicy;
	import java.lang.annotation.Target;
	
	@Target(ElementType.FIELD)
	@Retention(RetentionPolicy.RUNTIME)
	public @interface Id {
	}

	model/annotation/Positive.java

CODE	package model.annotation;
	
	import java.lang.annotation.ElementType;
	import java.lang.annotation.Retention;
	import java.lang.annotation.RetentionPolicy;
	import java.lang.annotation.Target;
	
	@Target(ElementType.FIELD)
	@Retention(RetentionPolicy.RUNTIME)
	public @interface Positive {
	  String message() default "must be > 0";
	}

	model/annotation/MaxLength.java
CODE	package model.annotation;

	import java.lang.annotation.ElementType;
	import java.lang.annotation.Retention;
	import java.lang.annotation.RetentionPolicy;
	import java.lang.annotation.Target;
	
	@Target(ElementType.FIELD)
	@Retention(RetentionPolicy.RUNTIME)
	public @interface MaxLength {
	  int value();
	}

	model/Account.java

CODE	package model;
	
	import java.util.Objects;
	import model.annotation.Id;
	import model.annotation.MaxLength;
	import model.annotation.Positive;
	
	public abstract class Account implements Transactable,
	InterestBearing {
	  @Id
	  private final String accountNumber;
	
	  @MaxLength(20)
	  private final String ownerName;
	
	  @Positive
	  private long balance;
	
	  private final boolean active;
	
	  private static int accountCounter = 0;
	
	  protected Account(String ownerName, long openingBalance) {
	  accountCounter++;
	  this.accountNumber = String.format("AC%04d",
	accountCounter);
	  this.ownerName = ownerName;
	  this.balance = openingBalance;
	  this.active = true;
	  }
	
	  public abstract boolean canWithdraw(long amount);
	
	  @Override
	  public void deposit(long amount) {
	  if (amount > 0) {
	  balance += amount;
	  }
	  }

```
@Override
public boolean withdraw(long amount) {
    if (amount <= 0 || !canWithdraw(amount)) {
        return false;
    }

    balance -= amount;
    return true;
}

public String getAccountNumber() {
    return accountNumber;
}

public String getOwnerName() {
    return ownerName;
}

@Override
public long getBalance() {
    return balance;
}

public boolean isActive() {
    return active;
}

@Override
public String toString() {
    return "Account Number: " + accountNumber
            + " | Owner: " + ownerName
            + " | Balance: Rs. " + balance
            + " | Active: " + active;
}

@Override
public boolean equals(Object object) {
    if (this == object) {
        return true;
    }

    if (!(object instanceof Account account)) {
        return false;
    }

    return accountNumber.equals(account.accountNumber);
}

@Override
```

	public int hashCode() {
	return Objects.hash(accountNumber);
	}
	}

	util/AnnotationValidator.java

CODE	package util;
	
	import java.lang.reflect.Field;
	import java.util.ArrayList;
	import java.util.List;
	import model.annotation.MaxLength;
	import model.annotation.Positive;
	
	public class AnnotationValidator {
	private AnnotationValidator() {
	}
	
	public static String[] validate(Object obj) {
	if (obj == null) {
	return new String[] {"Object cannot be null."};
	}
	
	List<String> errors = new ArrayList<>();
	
	for (Field field : obj.getClass().getDeclaredFields()) {
	field.setAccessible(true);
	
	try {
	Object value = field.get(obj);
	
	if (field.isAnnotationPresent(Positive.class)
	&& value instanceof Number number
	&& number.doubleValue() <= 0) {
	Positive rule = field.getAnnotation(Positive.class);
	errors.add(field.getName() + " " + rule.message());
	}
	
	if (field.isAnnotationPresent(MaxLength.class)
	&& value instanceof String text) {
	MaxLength rule =
	field.getAnnotation(MaxLength.class);
	
	if (text.length() > rule.value()) {
	errors.add(field.getName() + " must be at most "
	+ rule.value() + " characters");
	}
	}
	} catch (IllegalAccessException e) {
	errors.add("Could not validate field: " + field.getName());
	}

```
        }

        return errors.toArray(new String[0]);
    }
}
```

service/MiniBank.java

CODE

```
package service;

import model.Account;
import model.CurrentAccount;
import model.FixedDepositAccount;
import model.Premium;
import model.SavingsAccount;
import model.WithdrawRule;
import util.AnnotationValidator;

import static util.Validator.isValidMobile;

public class MiniBank {
    public static void main(String[] args) {
        Account[] accounts = {
            new SavingsAccount("Bhavya", 5000, 1000),
            new CurrentAccount("Japan", 10000, 2000),
            new FixedDepositAccount("Shivam", 20000)
        };

        System.out.println("MiniBank Accounts");
        System.out.println("==================");

        for (Account account : accounts) {
            System.out.println(account);
            System.out.println("Interest rate: " + account.interestRate() +
"%");
            System.out.println("Yearly interest: Rs. "
                    + account.yearlyInterest());

            if (account instanceof SavingsAccount savings) {
                System.out.println("Minimum balance: Rs. "
                        + savings.getMinBalance());
            }

            System.out.println();
        }

        WithdrawRule anonymousRule = new WithdrawRule() {
            @Override
            public boolean allow(Account account, long amount) {
                return account.canWithdraw(amount);
            }
        };
```

```
        WithdrawRule lambdaRule =
                (account, amount) -> account.canWithdraw(amount);

        System.out.println("Anonymous rule allows withdrawal: "
                + anonymousRule.allow(accounts[0], 1000));
        System.out.println("Lambda rule allows withdrawal: "
                + lambdaRule.allow(accounts[0], 1000));

        System.out.println("Bhavya has Premium status: "
                + (accounts[0] instanceof Premium));

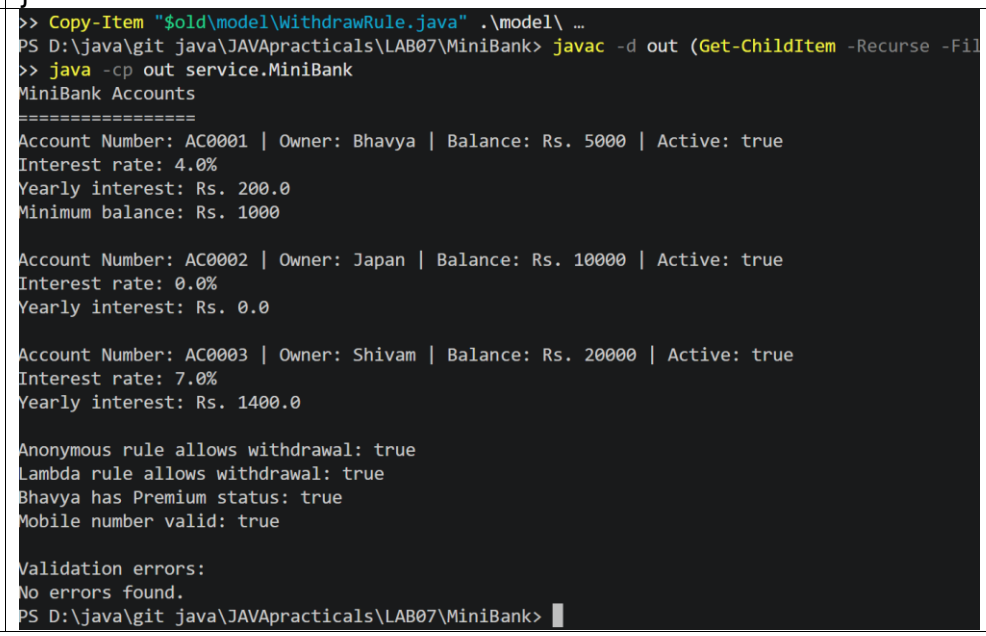
        System.out.println("Mobile number valid: "
                + isValidMobile("9876543210"));

        Account invalidAccount =
                new FixedDepositAccount("Test Customer", -500);

        System.out.println("\nValidation errors:");
        String[] errors = AnnotationValidator.validate(invalidAccount);

        if (errors.length == 0) {
            System.out.println("No errors found.");
        } else {
            for (String error : errors) {
                System.out.println(error);
            }
        }
    }
}
```

OUTPUT



Practical 8

	exception/BankException.java

CODE	package exception;
	
	public class BankException extends Exception {
	  public BankException(String message) {
	  super(message);
	  }
	}

	exception/InsufficientFundsException.java

CODE	package exception;
	
	public class InsufficientFundsException extends BankException {
	  private final long shortfall;
	
	  public InsufficientFundsException(String message, long shortfall)
	{
	  super(message);
	  this.shortfall = shortfall;
	  }
	
	  public long getShortfall() {
	  return shortfall;
	  }
	}

	exception/AccountNotFoundException.java

CODE	package exception;
	
	public class AccountNotFoundException extends BankException {
	  public AccountNotFoundException(String message) {
	  super(message);
	  }
	}

	exception/InvalidAmountException.java

CODE	package exception;
	
	public class InvalidAmountException extends BankException {
	  public InvalidAmountException(String message) {
	  super(message);
	  }
	}

	model/Transactable.java
CODE	package model;

	import exception.InsufficientFundsException;
	import exception.InvalidAmountException;
	
	public interface Transactable {
	  void deposit(long amount) throws InvalidAmountException;
	
	  boolean withdraw(long amount)
	  throws InsufficientFundsException, InvalidAmountException;
	}

	model/Account.java

CODE	package model;
	
	import exception.AccountNotFoundException;
	import exception.BankException;
	import exception.InsufficientFundsException;
	import exception.InvalidAmountException;
	import java.util.Objects;
	import model.annotation.Id;
	import model.annotation.MaxLength;
	import model.annotation.Positive;
	
	public abstract class Account implements Transactable,
	InterestBearing {
	  @Id
	  private final String accountNumber;
	
	  @MaxLength(20)
	  private final String ownerName;
	
	  @Positive
	  private long balance;
	
	  private final boolean active;
	  private static int accountCounter = 0;
	
	  protected Account(String ownerName, long openingBalance) {
	  accountCounter++;
	  this.accountNumber = String.format("AC%04d",
	accountCounter);
	  this.ownerName = ownerName;
	  this.balance = openingBalance;
	  this.active = true;
	  }
	
	  public abstract double interestRate();
	
	  public abstract boolean canWithdraw(long amount);
	
	  protected abstract long availableBalanceForWithdrawal();

```
    @Override
    public void deposit(long amount) throws InvalidAmountException
{
        if (amount <= 0) {
            throw new InvalidAmountException(
                    "Deposit amount must be greater than zero."
            );
        }

        balance += amount;
    }

    @Override
    public boolean withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException
{
        if (amount <= 0) {
            throw new InvalidAmountException(
                    "Withdrawal amount must be greater than zero."
            );
        }

        long available = availableBalanceForWithdrawal();
        if (amount > available) {
            long shortfall = amount - available;
            throw new InsufficientFundsException(
                    "Insufficient funds. Shortfall: Rs. " + shortfall,
                    shortfall
            );
        }

        balance -= amount;
        return true;
    }

    public void transfer(Account to, long amount) throws
BankException {
        try {
            if (to == null) {
                throw new AccountNotFoundException(
                        "Destination account was not found."
                );
            }

            withdraw(amount);
            to.deposit(amount);
            System.out.println("Transfer successful.");
        } catch (BankException e) {
            throw e;
```

```
        } finally {
            System.out.println("Transfer attempt finished.");
        }
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    @Override
    public long getBalance() {
        return balance;
    }

    public boolean isActive() {
        return active;
    }

    @Override
    public String toString() {
        return "Account Number: " + accountNumber
                + " | Owner: " + ownerName
                + " | Balance: Rs. " + balance
                + " | Active: " + active;
    }

    @Override
    public boolean equals(Object object) {
        if (this == object) {
            return true;
        }

        if (!(object instanceof Account account)) {
            return false;
        }

        return accountNumber.equals(account.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}
```

model/SavingsAccount.java

CODE

```
package model;
```

```
public class SavingsAccount extends Account implements Premium
{
    private final long minBalance;

    public SavingsAccount(String ownerName, long openingBalance,
long minBalance) {
        super(ownerName, openingBalance);
        this.minBalance = minBalance;
    }

    @Override
    public double interestRate() {
        return 4.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return amount > 0 && getBalance() - amount >= minBalance;
    }

    @Override
    protected long availableBalanceForWithdrawal() {
        return Math.max(0, getBalance() - minBalance);
    }

    public long getMinBalance() {
        return minBalance;
    }
}
```

model/CurrentAccount.java

CODE

```
package model;

public class CurrentAccount extends Account {
    private final long overdraftLimit;

    public CurrentAccount(String ownerName, long openingBalance,
long overdraftLimit) {
        super(ownerName, openingBalance);
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    public double interestRate() {
        return 0.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
```

	return amount > 0 && getBalance() - amount >= -
	overdraftLimit;
	}
	
	@Override
	protected long availableBalanceForWithdrawal() {
	return Math.max(0, getBalance() + overdraftLimit);
	}
	}

	model/FixedDepositAccount.java

CODE	package model;
	
	public class FixedDepositAccount extends Account {
	public FixedDepositAccount(String ownerName, long
	openingBalance) {
	super(ownerName, openingBalance);
	}
	
	@Override
	public double interestRate() {
	return 7.0;
	}
	
	@Override
	public boolean canWithdraw(long amount) {
	return false;
	}
	
	@Override
	protected long availableBalanceForWithdrawal() {
	return 0;
	}
	}

	service/MiniBank.java

CODE	package service;
	
	import exception.BankException;
	import exception.InsufficientFundsException;
	import exception.InvalidAmountException;
	import model.Account;
	import model.CurrentAccount;
	import model.FixedDepositAccount;
	import model.SavingsAccount;
	
	public class MiniBank {
	public static void main(String[] args) {
	Account savings = new SavingsAccount("Bhavya", 5000,
	1000);
	Account current = new CurrentAccount("Japan", 10000, 2000);

```
        Account fixedDeposit = new FixedDepositAccount("Shivam",
20000);

        try {
            savings.deposit(1000);
            System.out.println("Deposit successful. Balance: Rs. "
                    + savings.getBalance());

            savings.withdraw(8000);
        } catch (InsufficientFundsException e) {
            System.out.println("Withdrawal failed: " + e.getMessage());
            System.out.println("Shortfall: Rs. " + e.getShortfall());
        } catch (InvalidAmountException e) {
            System.out.println("Invalid amount: " + e.getMessage());
        } finally {
            System.out.println("Withdrawal operation finished.");
        }

        try {
            savings.transfer(current, 1000);
            System.out.println("Savings balance: Rs. " +
savings.getBalance());
            System.out.println("Current balance: Rs. " +
current.getBalance());
        } catch (BankException e) {
            System.out.println("Transfer failed: " + e.getMessage());
        }

        try {
            fixedDeposit.withdraw(500);
        } catch (InsufficientFundsException e) {
            System.out.println("Fixed deposit withdrawal failed: "
                    + e.getMessage());
        } catch (InvalidAmountException e) {
            System.out.println("Invalid amount: " + e.getMessage());
        }

        try (BankSession session = new BankSession()) {
            session.displayMessage();
        }
    }
}

class BankSession implements AutoCloseable {
    public void displayMessage() {
        System.out.println("Bank session is open.");
    }

    @Override
    public void close() {
```

	System.out.println("Bank session closed automatically."); } }
OUTPUT	

Practical 9

	model/Account.java
CODE	package model; import exception.AccountNotFoundException; import exception.BankException; import exception.InsufficientFundsException;

```
import exception.InvalidAmountException;
import java.util.Objects;
import model.annotation.Id;
import model.annotation.MaxLength;
import model.annotation.Positive;

public abstract class Account implements Transactable,
InterestBearing {
    @Id
    private final String accountNumber;

    @MaxLength(20)
    private final String ownerName;

    @Positive
    private long balance;

    private final boolean active;
    private static int accountCounter = 0;

    protected Account(String ownerName, long openingBalance) {
        accountCounter++;
        this.accountNumber = String.format("AC%04d",
accountCounter);
        this.ownerName = ownerName;
        this.balance = openingBalance;
        this.active = true;
    }

    public abstract double interestRate();

    public abstract boolean canWithdraw(long amount);

    protected abstract long availableBalanceForWithdrawal();

    @Override
    public synchronized void deposit(long amount) throws
InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException(
                    "Deposit amount must be greater than zero."
            );
        }

        balance += amount;
    }

    @Override
    public synchronized boolean withdraw(long amount)
```

```
        throws InsufficientFundsException, InvalidAmountException
{
    if (amount <= 0) {
        throw new InvalidAmountException(
                "Withdrawal amount must be greater than zero."
        );
    }

    long available = availableBalanceForWithdrawal();
    if (amount > available) {
        long shortfall = amount - available;
        throw new InsufficientFundsException(
                "Insufficient funds. Shortfall: Rs. " + shortfall,
                shortfall
        );
    }

    balance -= amount;
    return true;
  }

  public void transfer(Account to, long amount) throws
BankException {
    try {
        if (to == null) {
            throw new AccountNotFoundException(
                    "Destination account was not found."
            );
        }

        withdraw(amount);
        to.deposit(amount);
        System.out.println("Transfer successful.");
    } catch (BankException e) {
        throw e;
    } finally {
        System.out.println("Transfer attempt finished.");
    }
  }

  public String getAccountNumber() {
    return accountNumber;
  }

  public String getOwnerName() {
    return ownerName;
  }

  @Override
  public synchronized long getBalance() {
```

```
        return balance;
    }

    public boolean isActive() {
        return active;
    }

    @Override
    public String toString() {
        return "Account Number: " + accountNumber
                + " | Owner: " + ownerName
                + " | Balance: Rs. " + getBalance()
                + " | Active: " + active;
    }

    @Override
    public boolean equals(Object object) {
        if (this == object) {
            return true;
        }

        if (!(object instanceof Account account)) {
            return false;
        }

        return accountNumber.equals(account.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}
```

service/MiniBank.java

CODE

```
package service;

import exception.InvalidAmountException;
import model.Account;
import model.SavingsAccount;

public class MiniBank {
    public static void main(String[] args) {
        Account account = new SavingsAccount("Thread Test", 0, 0);

        int numberOfThreads = 10;
        int depositsPerThread = 10_000;
        long depositAmount = 1;

        Thread[] threads = new Thread[numberOfThreads];
```

```
        System.out.println("Starting deposit threads...");

        for (int i = 0; i < numberOfThreads; i++) {
            AccountWorker worker =
                    new AccountWorker(account, depositsPerThread,
depositAmount);

            threads[i] = new Thread(worker, "Deposit-Thread-" + (i + 1));
            threads[i].start();
        }

        for (Thread thread : threads) {
            try {
                thread.join();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                System.out.println("Main thread was interrupted.");
                return;
            }
        }

        long expectedBalance =
                (long) numberOfThreads * depositsPerThread *
depositAmount;

        System.out.println("\nAll threads have finished.");
        System.out.println("Expected balance: Rs. " +
expectedBalance);
        System.out.println("Actual balance:   Rs. " +
account.getBalance());
    }
}

class AccountWorker implements Runnable {
    private final Account account;
    private final int times;
    private final long amount;

    public AccountWorker(Account account, int times, long amount) {
        this.account = account;
        this.times = times;
        this.amount = amount;
    }

    @Override
    public void run() {
        String threadName = Thread.currentThread().getName();
        System.out.println(threadName + " started.");

        for (int i = 0; i < times; i++) {
```

```
            try {
                account.deposit(amount);
            } catch (InvalidAmountException e) {
                System.out.println(threadName + ": " + e.getMessage());
                return;
            }
        }

        System.out.println(threadName + " finished.");
    }
}
```

OUTPUT

```
>> Copy-Item "$previous\exception\*.java" .\exception\ …
PS D:\java\git java\JAVApracticals\LAB09\MiniBank> javac -d
>> if ($LASTEXITCODE -eq 0) { java -cp out service.MiniBank
Starting deposit threads...
Deposit-Thread-2 started.
Deposit-Thread-6 started.
Deposit-Thread-7 started.
Deposit-Thread-5 started.
Deposit-Thread-3 started.
Deposit-Thread-1 started.
Deposit-Thread-4 started.
Deposit-Thread-8 started.
Deposit-Thread-9 started.
Deposit-Thread-10 started.
Deposit-Thread-2 finished.
Deposit-Thread-6 finished.
Deposit-Thread-7 finished.
Deposit-Thread-3 finished.
Deposit-Thread-1 finished.
Deposit-Thread-4 finished.
Deposit-Thread-10 finished.
Deposit-Thread-8 finished.
Deposit-Thread-9 finished.
Deposit-Thread-5 finished.

All threads have finished.
Expected balance: Rs. 100000
Actual balance:   Rs. 100000
PS D:\java\git java\JAVApracticals\LAB09\MiniBank>
```

Practical 10

	model/Account.java

CODE	package model;
	
	import exception.AccountNotFoundException;
	import exception.BankException;
	import exception.InsufficientFundsException;
	import exception.InvalidAmountException;
	import java.util.Objects;
	import model.annotation.Id;
	import model.annotation.MaxLength;
	import model.annotation.Positive;
	
	public abstract class Account implements Transactable,
	InterestBearing {
	@Id
	private final String accountNumber;
	
	@MaxLength(20)
	private final String ownerName;
	
	@Positive
	private long balance;
	
	private final boolean active;
	private static int accountCounter = 0;
	
	protected Account(String ownerName, long openingBalance) {
	accountCounter++;
	this.accountNumber = String.format("AC%04d",
	accountCounter);
	this.ownerName = ownerName;
	this.balance = openingBalance;
	this.active = true;
	}
	
	public abstract double interestRate();

```
    public abstract boolean canWithdraw(long amount);

    protected abstract long availableBalanceForWithdrawal();

    @Override
    public synchronized void deposit(long amount) throws
InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException(
                    "Deposit amount must be greater than zero."
            );
        }

        balance += amount;
    }

    @Override
    public synchronized boolean withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException
{
        if (amount <= 0) {
            throw new InvalidAmountException(
                    "Withdrawal amount must be greater than zero."
            );
        }

        long available = availableBalanceForWithdrawal();

        if (amount > available) {
            long shortfall = amount - available;
            throw new InsufficientFundsException(
                    "Insufficient funds. Shortfall: Rs. " + shortfall,
                    shortfall
            );
        }

        balance -= amount;
        return true;
    }

    public void transfer(Account to, long amount) throws
BankException {
        if (to == null) {
            throw new AccountNotFoundException(
                    "Destination account was not found."
            );
        }

        // Both directions lock the accounts in the same order.
        Account first = this;
```

```
        Account second = to;

        if
(first.getAccountNumber().compareTo(second.getAccountNumber())
> 0) {
            first = to;
            second = this;
        }

        try {
            synchronized (first) {
                synchronized (second) {
                    withdraw(amount);
                    to.deposit(amount);
                    System.out.println("Transfer successful.");
                }
            }
        } catch (BankException e) {
            throw e;
        } finally {
            System.out.println("Transfer attempt finished.");
        }
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    @Override
    public synchronized long getBalance() {
        return balance;
    }

    public boolean isActive() {
        return active;
    }

    @Override
    public double yearlyInterest() {
        return getBalance() * interestRate() / 100.0;
    }

    @Override
    public String toString() {
        return "Account Number: " + accountNumber
                + " | Owner: " + ownerName
```

```
                + " | Balance: Rs. " + getBalance()
                + " | Active: " + active;
    }

    @Override
    public boolean equals(Object object) {
        if (this == object) {
            return true;
        }

        if (!(object instanceof Account account)) {
            return false;
        }

        return accountNumber.equals(account.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}
```

model/Transactable.java

CODE

```
package model;

import exception.InsufficientFundsException;
import exception.InvalidAmountException;

public interface Transactable {
    void deposit(long amount) throws InvalidAmountException;

    boolean withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException;
}
```

model/SavingsAccount.java

CODE

```
package model;

public class SavingsAccount extends Account implements Premium
{
    private final long minBalance;

    public SavingsAccount(String ownerName, long openingBalance,
long minBalance) {
        super(ownerName, openingBalance);
        this.minBalance = minBalance;
    }

    @Override
    public double interestRate() {
        return 4.0;
```

	}
	
	@Override
	public boolean canWithdraw(long amount) {
	return amount > 0 && getBalance() - amount >= minBalance;
	}
	
	@Override
	protected long availableBalanceForWithdrawal() {
	return Math.max(0, getBalance() - minBalance);
	}
	
	public long getMinBalance() {
	return minBalance;
	}
	}

	model/CurrentAccount.java

CODE	package model;
	
	public class CurrentAccount extends Account {
	private final long overdraftLimit;
	
	public CurrentAccount(String ownerName, long openingBalance, long overdraftLimit) {
	super(ownerName, openingBalance);
	this.overdraftLimit = overdraftLimit;
	}
	
	@Override
	public double interestRate() {
	return 0.0;
	}
	
	@Override
	public boolean canWithdraw(long amount) {
	return amount > 0 && getBalance() - amount >= -overdraftLimit;
	}
	
	@Override
	protected long availableBalanceForWithdrawal() {
	return Math.max(0, getBalance() + overdraftLimit);
	}
	}

	model/FixedDepositAccount.java

CODE	package model;
	
	public class FixedDepositAccount extends Account {
	public FixedDepositAccount(String ownerName, long openingBalance) {

```
        super(ownerName, openingBalance);
    }

    @Override
    public double interestRate() {
        return 7.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return false;
    }

    @Override
    protected long availableBalanceForWithdrawal() {
        return 0;
    }
}
```

service/MiniBank.java

CODE

```
package service;

import exception.BankException;
import exception.InsufficientFundsException;
import exception.InvalidAmountException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import model.Account;
import model.CurrentAccount;
import model.SavingsAccount;

public class MiniBank {
    public static void main(String[] args) throws InterruptedException
{
        Account account = new SavingsAccount("Bhavya", 10000, 0);

        // Submit deposit and withdrawal tasks to a four-thread pool.
        TransactionProcessor processor = new
TransactionProcessor();

        for (int i = 0; i < 10; i++) {
            processor.submit(new TransactionTask(account, true, 100));
            processor.submit(new TransactionTask(account, false, 50));
        }

        processor.stop();
        System.out.println("Balance after thread-pool tasks: Rs. "
                + account.getBalance());

        // Producer-consumer example using a shared array buffer.
```

```
        TransactionBuffer buffer = new TransactionBuffer(3);

        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    buffer.put(new TransactionTask(account, true, 10));
                    System.out.println("Produced transaction " + i);
                }

                buffer.finish();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }, "Producer");

        Thread consumer = new Thread(() -> {
            try {
                Runnable task;

                while ((task = buffer.take()) != null) {
                    task.run();
                    System.out.println("Consumer processed a
transaction.");
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }, "Consumer");

        producer.start();
        consumer.start();
        producer.join();
        consumer.join();

        System.out.println("Balance after producer-consumer: Rs. "
                + account.getBalance());

        // Opposite-direction transfers acquire locks in account-number
order.
        Account accountA = new CurrentAccount("Account A", 1000,
0);
        Account accountB = new CurrentAccount("Account B", 1000,
0);

        Thread transferAtoB = new Thread(
                () -> doTransfer(accountA, accountB, 100),
                "Transfer-A-to-B"
        );

        Thread transferBtoA = new Thread(
```

```
            () -> doTransfer(accountB, accountA, 100),
            "Transfer-B-to-A"
        );

        transferAtoB.start();
        transferBtoA.start();
        transferAtoB.join();
        transferBtoA.join();

        System.out.println("Account A balance: Rs. " +
accountA.getBalance());
        System.out.println("Account B balance: Rs. " +
accountB.getBalance());
    }

    private static void doTransfer(Account from, Account to, long
amount) {
        try {
            System.out.println(Thread.currentThread().getName() + "
started.");
            from.transfer(to, amount);
            System.out.println(Thread.currentThread().getName() + "
finished.");
        } catch (BankException e) {
            System.out.println("Transfer failed: " + e.getMessage());
        }
    }
}

class TransactionProcessor {
    private final ExecutorService executor =
            Executors.newFixedThreadPool(4);

    public void submit(Runnable task) {
        executor.execute(task);
    }

    public void stop() throws InterruptedException {
        executor.shutdown();

        if (!executor.awaitTermination(30, TimeUnit.SECONDS)) {
            executor.shutdownNow();
            executor.awaitTermination(30, TimeUnit.SECONDS);
        }
    }
}

class TransactionTask implements Runnable {
    private final Account account;
    private final boolean deposit;
```

```
    private final long amount;

    public TransactionTask(Account account, boolean deposit, long
amount) {
        this.account = account;
        this.deposit = deposit;
        this.amount = amount;
    }

    @Override
    public void run() {
        String threadName = Thread.currentThread().getName();

        try {
            if (deposit) {
                account.deposit(amount);
                System.out.println(threadName + " deposited Rs. " +
amount);
            } else {
                account.withdraw(amount);
                System.out.println(threadName + " withdrew Rs. " +
amount);
            }
        } catch (InsufficientFundsException | InvalidAmountException
e) {
            System.out.println(threadName + ": " + e.getMessage());
        }
    }
}

class TransactionBuffer {
    private final Runnable[] items;
    private int head;
    private int tail;
    private int count;
    private boolean finished;

    public TransactionBuffer(int capacity) {
        items = new Runnable[capacity];
    }

    public synchronized void put(Runnable task) throws
InterruptedException {
        while (count == items.length) {
            wait();
        }

        items[tail] = task;
        tail = (tail + 1) % items.length;
        count++;
```

```
        notifyAll();
    }

    public synchronized Runnable take() throws InterruptedException
{
        while (count == 0 && !finished) {
            wait();
        }

        if (count == 0 && finished) {
            return null;
        }

        Runnable task = items[head];
        items[head] = null;
        head = (head + 1) % items.length;
        count--;
        notifyAll();
        return task;
    }

    public synchronized void finish() {
        finished = true;
        notifyAll();
    }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB10\M
>> if ($LASTEXITCODE -eq 0) { java -cp out
pool-1-thread-1 deposited Rs. 100
pool-1-thread-3 deposited Rs. 100
pool-1-thread-2 withdrew Rs. 50
pool-1-thread-4 withdrew Rs. 50
pool-1-thread-4 withdrew Rs. 50
pool-1-thread-1 deposited Rs. 100
pool-1-thread-1 withdrew Rs. 50
pool-1-thread-3 withdrew Rs. 50
pool-1-thread-2 deposited Rs. 100
pool-1-thread-4 deposited Rs. 100
pool-1-thread-1 deposited Rs. 100
pool-1-thread-3 withdrew Rs. 50
pool-1-thread-2 deposited Rs. 100
pool-1-thread-4 withdrew Rs. 50
pool-1-thread-1 deposited Rs. 100
pool-1-thread-3 withdrew Rs. 50
pool-1-thread-2 deposited Rs. 100
pool-1-thread-4 withdrew Rs. 50
pool-1-thread-1 deposited Rs. 100
pool-1-thread-3 withdrew Rs. 50
Balance after thread-pool tasks: Rs. 10500
Consumer deposited Rs. 10
Consumer processed a transaction.
Produced transaction 1
Produced transaction 2
Consumer deposited Rs. 10
Consumer processed a transaction.
Produced transaction 3
Consumer deposited Rs. 10
Consumer processed a transaction.
Produced transaction 4
Consumer deposited Rs. 10
Consumer processed a transaction.
Consumer deposited Rs. 10
Consumer processed a transaction.
Produced transaction 4
Consumer deposited Rs. 10
Consumer processed a transaction.
Consumer deposited Rs. 10
Consumer processed a transaction.
Produced transaction 5
Balance after producer-consumer: Rs. 10550
Transfer-B-to-A started.
Transfer-A-to-B started.
Transfer successful.
Transfer attempt finished.
Transfer successful.
Transfer-B-to-A finished.
Transfer attempt finished.
Transfer-A-to-B finished.
Account A balance: Rs. 1000
Account B balance: Rs. 1000
```

Practical 11

	exception/BankException.java

CODE	package exception;
	
	public class BankException extends Exception {
	  public BankException(String message) {
	  super(message);
	  }
	}

	exception/AccountNotFoundException.java

CODE	package exception;
	
	public class AccountNotFoundException extends BankException {
	  public AccountNotFoundException(String message) {
	  super(message);
	  }
	}

	exception/InsufficientFundsException.java

CODE	package exception;
	
	public class InsufficientFundsException extends BankException {
	  private final long shortfall;
	
	  public InsufficientFundsException(String message, long shortfall) {
	  super(message);
	  this.shortfall = shortfall;
	  }
	
	  public long getShortfall() {
	  return shortfall;
	  }
	}

	exception/InvalidAmountException.java

CODE	package exception;
	
	public class InvalidAmountException extends BankException {
	  public InvalidAmountException(String message) {
	  super(message);
	  }
	}

	model/annotation/Id.java
CODE	package model.annotation; import java.lang.annotation.ElementType; import java.lang.annotation.Retention; import java.lang.annotation.RetentionPolicy; import java.lang.annotation.Target;

	@Target(ElementType.FIELD)
	@Retention(RetentionPolicy.RUNTIME)
	public @interface Id {
	}

	model/annotation/MaxLength.java

CODE	package model.annotation;
	
	import java.lang.annotation.ElementType;
	import java.lang.annotation.Retention;
	import java.lang.annotation.RetentionPolicy;
	import java.lang.annotation.Target;
	
	@Target(ElementType.FIELD)
	@Retention(RetentionPolicy.RUNTIME)
	public @interface MaxLength {
	  int value();
	}

	model/annotation/Positive.java

CODE	package model.annotation;
	
	import java.lang.annotation.ElementType;
	import java.lang.annotation.Retention;
	import java.lang.annotation.RetentionPolicy;
	import java.lang.annotation.Target;
	
	@Target(ElementType.FIELD)
	@Retention(RetentionPolicy.RUNTIME)
	public @interface Positive {
	  String message() default "must be > 0";
	}

	model/Transactable.java

CODE	package model;
	
	import exception.InsufficientFundsException;
	import exception.InvalidAmountException;
	
	public interface Transactable {
	  void deposit(long amount) throws InvalidAmountException;
	
	  boolean withdraw(long amount)
	  throws InsufficientFundsException, InvalidAmountException;
	}

	model/InterestBearing.java

CODE	package model;
	
	public interface InterestBearing {
	  double interestRate();
	
	  long getBalance();

```
    default double yearlyInterest() {
        return getBalance() * interestRate() / 100.0;
    }
}
```

model/Premium.java

CODE

```
package model;

public interface Premium {
}
```

model/Account.java

CODE

```
package model;

import exception.AccountNotFoundException;
import exception.BankException;
import exception.InsufficientFundsException;
import exception.InvalidAmountException;
import java.io.Serializable;
import java.util.Objects;
import model.annotation.Id;
import model.annotation.MaxLength;
import model.annotation.Positive;

public abstract class Account
        implements Transactable, InterestBearing, Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    private final String accountNumber;

    @MaxLength(20)
    private final String ownerName;

    @Positive
    private long balance;

    private final boolean active;
    private static int accountCounter = 0;

    protected Account(String ownerName, long openingBalance) {
        accountCounter++;
        this.accountNumber = String.format("AC%04d", accountCounter);
        this.ownerName = ownerName;
        this.balance = openingBalance;
        this.active = true;
    }

    public abstract boolean canWithdraw(long amount);
```

```
    protected abstract long availableBalanceForWithdrawal();

    @Override
    public synchronized void deposit(long amount) throws
InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException(
                    "Deposit amount must be greater than zero."
            );
        }

        balance += amount;
    }

    @Override
    public synchronized boolean withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException(
                    "Withdrawal amount must be greater than zero."
            );
        }

        long available = availableBalanceForWithdrawal();

        if (amount > available) {
            long shortfall = amount - available;
            throw new InsufficientFundsException(
                    "Insufficient funds. Shortfall: Rs. " + shortfall,
                    shortfall
            );
        }

        balance -= amount;
        return true;
    }

    public void transfer(Account to, long amount) throws BankException {
        if (to == null) {
            throw new AccountNotFoundException(
                    "Destination account was not found."
            );
        }

        // Lock accounts in the same order to avoid deadlock.
        Account first = this;
        Account second = to;
```

```
        if
(first.getAccountNumber().compareTo(second.getAccountNumber()) > 0)
{
            first = to;
            second = this;
        }

        try {
            synchronized (first) {
                synchronized (second) {
                    withdraw(amount);
                    to.deposit(amount);
                    System.out.println("Transfer successful.");
                }
            }
        } catch (BankException e) {
            throw e;
        } finally {
            System.out.println("Transfer attempt finished.");
        }
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    @Override
    public synchronized long getBalance() {
        return balance;
    }

    public boolean isActive() {
        return active;
    }

    @Override
    public String toString() {
        return "Account Number: " + accountNumber
                + " | Owner: " + ownerName
                + " | Balance: Rs. " + getBalance()
                + " | Active: " + active;
    }

    @Override
    public boolean equals(Object object) {
        if (this == object) {
```

```
            return true;
        }

        if (!(object instanceof Account account)) {
            return false;
        }

        return accountNumber.equals(account.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}
```

model/SavingsAccount.java

CODE

```
package model;

import java.io.Serializable;

public class SavingsAccount extends Account implements Premium,
Serializable {
    private static final long serialVersionUID = 1L;

    private final long minBalance;

    public SavingsAccount(String ownerName, long openingBalance, long
minBalance) {
        super(ownerName, openingBalance);
        this.minBalance = minBalance;
    }

    @Override
    public double interestRate() {
        return 4.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return amount > 0 && getBalance() - amount >= minBalance;
    }

    @Override
    protected long availableBalanceForWithdrawal() {
        return Math.max(0, getBalance() - minBalance);
    }

    public long getMinBalance() {
        return minBalance;
    }
```

	}
	model/CurrentAccount.java

CODE	package model;
	
	import java.io.Serializable;
	
	public class CurrentAccount extends Account implements Serializable {
	  private static final long serialVersionUID = 1L;
	
	  private final long overdraftLimit;
	
	  public CurrentAccount(String ownerName, long openingBalance, long overdraftLimit) {
	  super(ownerName, openingBalance);
	  this.overdraftLimit = overdraftLimit;
	  }
	
	  @Override
	  public double interestRate() {
	  return 0.0;
	  }
	
	  @Override
	  public boolean canWithdraw(long amount) {
	  return amount > 0 && getBalance() - amount >= -overdraftLimit;
	  }
	
	  @Override
	  protected long availableBalanceForWithdrawal() {
	  return Math.max(0, getBalance() + overdraftLimit);
	  }
	}

	model/FixedDepositAccount.java

CODE	package model;
	
	import java.io.Serializable;
	
	public class FixedDepositAccount extends Account implements Serializable {
	  private static final long serialVersionUID = 1L;
	
	  public FixedDepositAccount(String ownerName, long openingBalance) {
	  super(ownerName, openingBalance);
	  }
	
	  @Override
	  public double interestRate() {
	  return 7.0;
	  }

```
    @Override
    public boolean canWithdraw(long amount) {
        return false;
    }

    @Override
    protected long availableBalanceForWithdrawal() {
        return 0;
    }
}
```

model/Customer.java

CODE

```
package model;

import java.io.Serializable;

public class Customer implements Cloneable, Serializable {
    private static final long serialVersionUID = 1L;

    private String name;
    private String email;
    private String mobile;
    private final String customerId;
    private Address address;

    private static long customerCounter = 100;

    public Customer(String name, String email, String mobile, Address
address) {
        this.name = name;
        this.email = email;
        this.mobile = mobile;
        this.address = address;
        this.customerId = generateCustomerId();
    }

    private static String generateCustomerId() {
        customerCounter++;
        return "CUST" + customerCounter;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getMobile() {
```

```
    return mobile;
}

public String getCustomerId() {
    return customerId;
}

public Address getAddress() {
    return address;
}

@Override
public Customer clone() {
    try {
        return (Customer) super.clone();
    } catch (CloneNotSupportedException e) {
        throw new AssertionError(e);
    }
}

public static class Address implements Serializable {
    private static final long serialVersionUID = 1L;

    private final String line;
    private final String city;
    private final String pincode;

    public Address(String line, String city, String pincode) {
        this.line = line;
        this.city = city;
        this.pincode = pincode;
    }

    public String getLine() {
        return line;
    }

    public String getCity() {
        return city;
    }

    public String getPincode() {
        return pincode;
    }

    @Override
    public String toString() {
        return line + ", " + city + " - " + pincode;
    }
}
```

	}
	util/StatePersister.java

CODE	package util;
	
	import java.io.IOException;
	import java.io.ObjectInputStream;
	import java.io.ObjectOutputStream;
	import java.nio.file.Files;
	import java.nio.file.Path;
	import model.Account;
	
	public class StatePersister {
	  private StatePersister() {
	  }
	
	  public static void save(Account[] accounts, Path file) throws
	IOException {
	  Path parent = file.getParent();
	  if (parent != null) {
	  Files.createDirectories(parent);
	  }
	
	  try (ObjectOutputStream output = new ObjectOutputStream(
	  Files.newOutputStream(file))) {
	  output.writeObject(accounts);
	  }
	  }
	
	  public static Account[] load(Path file)
	  throws IOException, ClassNotFoundException {
	  try (ObjectInputStream input = new ObjectInputStream(
	  Files.newInputStream(file))) {
	  return (Account[]) input.readObject();
	  }
	  }
	}

	util/TransactionLog.java

CODE	package util;
	
	import java.io.IOException;
	import java.nio.charset.StandardCharsets;
	import java.nio.file.Files;
	import java.nio.file.Path;
	import java.nio.file.StandardOpenOption;
	
	public class TransactionLog {
	  private TransactionLog() {
	  }
	
	  public static void append(Path file, String line) throws IOException {

```
        Path parent = file.getParent();
        if (parent != null) {
            Files.createDirectories(parent);
        }

        Files.write(
                file,
                (line +
System.lineSeparator()).getBytes(StandardCharsets.UTF_8),
                StandardOpenOption.CREATE,
                StandardOpenOption.APPEND
        );
    }
}
```

util/ReportGenerator.java

CODE

```
package util;

import java.io.BufferedReader;
import java.io.IOException;
import java.nio.file.DirectoryStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.nio.file.attribute.BasicFileAttributes;

public class ReportGenerator {
    private ReportGenerator() {
    }

    public static String generate(Path logsFolder, Path reportFile)
            throws IOException {
        long totalDeposits = 0;
        long totalWithdrawals = 0;
        StringBuilder report = new StringBuilder();

        report.append("MiniBank End-of-Day Report\n");
        report.append("==========================\n");

        try (DirectoryStream<Path> entries =
                     Files.newDirectoryStream(logsFolder)) {
            for (Path file : entries) {
                if (!Files.isRegularFile(file)
                        || !file.getFileName().toString().endsWith(".log")) {
                    continue;
                }

                BasicFileAttributes attributes = Files.readAttributes(
                        file, BasicFileAttributes.class
                );
```

```
            if (attributes.size() == 0) {
                continue;
            }

            report.append("\nFile: ")
                    .append(file.getFileName())
                    .append(" | Size: ")
                    .append(attributes.size())
                    .append(" bytes | Last modified: ")
                    .append(attributes.lastModifiedTime())
                    .append("\n");

            try (BufferedReader reader = Files.newBufferedReader(file)) {
                String line;

                while ((line = reader.readLine()) != null) {
                    String[] parts = line.trim().split("\\s+");

                    if (parts.length != 2) {
                        continue;
                    }

                    try {
                        long amount = Long.parseLong(parts[1]);
                        if (amount <= 0) {
                            continue;
                        }

                        if (parts[0].equalsIgnoreCase("DEPOSIT")) {
                            totalDeposits += amount;
                        } else if (parts[0].equalsIgnoreCase("WITHDRAW")) {
                            totalWithdrawals += amount;
                        }
                    } catch (NumberFormatException ignored) {
                        // Ignore malformed log lines.
                    }
                }
            }
        }
    }

    report.append("\nTotal deposits: Rs. ")
            .append(totalDeposits)
            .append("\nTotal withdrawals: Rs. ")
            .append(totalWithdrawals)
            .append("\n");

    Path parent = reportFile.getParent();
    if (parent != null) {
        Files.createDirectories(parent);
```

```
        }

        Files.writeString(
                reportFile,
                report.toString(),
                StandardOpenOption.CREATE,
                StandardOpenOption.TRUNCATE_EXISTING
        );

        return report.toString();
    }
}
```

service/MiniBank.java

CODE

```
package service;

import java.nio.file.Files;
import java.nio.file.Path;
import model.Account;
import model.CurrentAccount;
import model.FixedDepositAccount;
import model.SavingsAccount;
import util.ReportGenerator;
import util.StatePersister;
import util.TransactionLog;

public class MiniBank {
    public static void main(String[] args) throws Exception {
        Account[] accounts = {
            new SavingsAccount("Bhavya", 5000, 1000),
            new CurrentAccount("Japan", 10000, 2000),
            new FixedDepositAccount("Shivam", 20000)
        };

        Path stateFile = Path.of("data", "accounts.dat");
        StatePersister.save(accounts, stateFile);

        Account[] loadedAccounts = StatePersister.load(stateFile);

        System.out.println("Reloaded accounts:");
        for (Account account : loadedAccounts) {
            System.out.println(account);
        }

        Path logsFolder = Path.of("logs");
        Files.createDirectories(logsFolder);

        Path transactionFile = logsFolder.resolve("transactions.log");
        TransactionLog.append(transactionFile, "DEPOSIT 500");
        TransactionLog.append(transactionFile, "WITHDRAW 200");
        TransactionLog.append(transactionFile, "DEPOSIT 1000");
```

```
        Path reportFile = Path.of("reports", "eod-summary.txt");
        String report = ReportGenerator.generate(logsFolder, reportFile);

        System.out.println("\n" + report);
        System.out.println("Report saved to: " +
reportFile.toAbsolutePath());
    }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals> cd "D:\java\git java\JAVApracticals\LAB11\M
>> javac -d out (Get-ChildItem -Recurse -Filter *.java | ForEach-Object FullNam
>> if ($LASTEXITCODE -eq 0) { java -cp out service.MiniBank }
Reloaded accounts:
Account Number: AC0001 | Owner: Bhavya | Balance: Rs. 5000 | Active: true
Account Number: AC0002 | Owner: Japan | Balance: Rs. 10000 | Active: true
Account Number: AC0003 | Owner: Shivam | Balance: Rs. 20000 | Active: true

MiniBank End-of-Day Report
==========================

File: transactions.log | Size: 41 bytes | Last modified: 2026-10-29T15:27:03.57

Total deposits: Rs. 1500
Total withdrawals: Rs. 200

Report saved to: D:\java\git java\JAVApracticals\LAB11\MiniBank\reports\eod-sum
PS D:\java\git java\JAVApracticals\LAB11\MiniBank>
```

Practical 12

	model/Account.java

CODE	package model;
	
	import exception.AccountNotFoundException;
	import exception.BankException;
	import exception.InsufficientFundsException;
	import exception.InvalidAmountException;
	import java.io.Serializable;
	import java.util.Objects;
	import model.annotation.Id;
	import model.annotation.MaxLength;
	import model.annotation.Positive;
	
	public abstract class Account
	implements Transactable, InterestBearing, Serializable,
	Comparable<Account> {
	
	private static final long serialVersionUID = 1L;
	
	@Id
	private final String accountNumber;
	
	@MaxLength(20)
	private final String ownerName;
	
	@Positive
	private long balance;
	
	private final boolean active;
	private static int accountCounter = 0;
	
	protected Account(String ownerName, long openingBalance) {
	accountCounter++;
	this.accountNumber = String.format("AC%04d",
	accountCounter);
	this.ownerName = ownerName;
	this.balance = openingBalance;
	this.active = true;
	}

```
    public abstract boolean canWithdraw(long amount);

    protected abstract long availableBalanceForWithdrawal();

    @Override
    public synchronized void deposit(long amount) throws
InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException(
                    "Deposit amount must be greater than zero."
            );
        }

        balance += amount;
    }

    @Override
    public synchronized boolean withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException(
                    "Withdrawal amount must be greater than zero."
            );
        }

        long available = availableBalanceForWithdrawal();

        if (amount > available) {
            long shortfall = amount - available;
            throw new InsufficientFundsException(
                    "Insufficient funds. Shortfall: Rs. " + shortfall,
                    shortfall
            );
        }

        balance -= amount;
        return true;
    }

    public void transfer(Account to, long amount) throws
BankException {
        if (to == null) {
            throw new AccountNotFoundException(
                    "Destination account was not found."
            );
        }

        Account first = this;
        Account second = to;
```

```
        if
(first.getAccountNumber().compareTo(second.getAccountNumber()) >
0) {
            first = to;
            second = this;
        }

        try {
            synchronized (first) {
                synchronized (second) {
                    withdraw(amount);
                    to.deposit(amount);
                    System.out.println("Transfer successful.");
                }
            }
        } catch (BankException e) {
            throw e;
        } finally {
            System.out.println("Transfer attempt finished.");
        }
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    @Override
    public synchronized long getBalance() {
        return balance;
    }

    public boolean isActive() {
        return active;
    }

    @Override
    public double compareTo(Account other) {
        return accountNumber.compareTo(other.accountNumber);
    }

    @Override
    public String toString() {
        return "Account Number: " + accountNumber
                + " | Owner: " + ownerName
                + " | Balance: Rs. " + getBalance()
                + " | Active: " + active;
```

```
    }

    @Override
    public boolean equals(Object object) {
        if (this == object) {
            return true;
        }

        if (!(object instanceof Account account)) {
            return false;
        }

        return accountNumber.equals(account.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}
```

service/AccountStore.java

CODE

```
package service;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import model.Account;

public class AccountStore {
    private final HashMap<String, Account> accounts = new
HashMap<>();
    private final ArrayList<String> transactionHistory = new
ArrayList<>();

    public void add(Account account) {
        if (account == null) {
            throw new IllegalArgumentException("Account cannot be
null.");
        }

        accounts.put(account.getAccountNumber(), account);
    }

    public Account get(String accountNumber) {
        return accounts.get(accountNumber);
    }

    public ArrayList<Account> getAll() {
        return new ArrayList<>(accounts.values());
    }
```

```
    public void addTransaction(String transaction) {
        transactionHistory.add(transaction);
    }

    public ArrayList<String> getTransactionHistory() {
        return new ArrayList<>(transactionHistory);
    }

    public ArrayList<Account> sortedByBalance() {
        ArrayList<Account> sorted = getAll();
        sorted.sort(Comparator.comparingLong(Account::getBalance));
        return sorted;
    }

    public ArrayList<Account> getAccountsAbove(long amount) {
        ArrayList<Account> result = new ArrayList<>();

        for (Account account : accounts.values()) {
            if (account.getBalance() > amount) {
                result.add(account);
            }
        }

        return result;
    }
}
```

service/TransactionProcessor.java

CODE

```
package service;

import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import model.Account;

public class TransactionProcessor {
    private final ExecutorService executor =
            Executors.newFixedThreadPool(4);

    private final ConcurrentHashMap<String, Account> accounts =
            new ConcurrentHashMap<>();

    public void addAccount(Account account) {
        accounts.put(account.getAccountNumber(), account);
    }

    public Account getAccount(String accountNumber) {
        return accounts.get(accountNumber);
    }
```

```
    public void submit(Runnable task) {
        executor.execute(task);
    }

    public void stop() throws InterruptedException {
        executor.shutdown();

        if (!executor.awaitTermination(30, TimeUnit.SECONDS)) {
            executor.shutdownNow();
            executor.awaitTermination(30, TimeUnit.SECONDS);
        }
    }
}
```

service/MiniBank.java

CODE

```
package service;

import exception.InsufficientFundsException;
import exception.InvalidAmountException;
import model.Account;
import model.CurrentAccount;
import model.FixedDepositAccount;
import model.SavingsAccount;

public class MiniBank {
    public static void main(String[] args) throws InterruptedException {
        AccountStore store = new AccountStore();

        Account account1 = new SavingsAccount("Bhavya", 5000,
1000);
        Account account2 = new CurrentAccount("Japan", 10000, 2000);
        Account account3 = new FixedDepositAccount("Shivam",
20000);

        store.add(account1);
        store.add(account2);
        store.add(account3);

        store.addTransaction("Bhavya opened account " +
account1.getAccountNumber());
        store.addTransaction("Japan opened account " +
account2.getAccountNumber());
        store.addTransaction("Shivam opened account " +
account3.getAccountNumber());

        System.out.println("Accounts sorted by balance:");
        for (Account account : store.sortedByBalance()) {
            System.out.println(account);
        }
```

```
        System.out.println("\nFind account by number:");
        Account found = store.get(account1.getAccountNumber());
        System.out.println(found);

        System.out.println("\nTransaction history:");
        for (String transaction : store.getTransactionHistory()) {
            System.out.println(transaction);
        }

        // Give the transaction processor its thread-safe account map.
        TransactionProcessor processor = new TransactionProcessor();
        for (Account account : store.getAll()) {
            processor.addAccount(account);
        }

        processor.submit(new AccountTransactionTask(
                processor, account1.getAccountNumber(), true, 500
        ));
        processor.submit(new AccountTransactionTask(
                processor, account2.getAccountNumber(), false, 1000
        ));

        processor.stop();

        System.out.println("\nBalances after concurrent transactions:");
        for (Account account : store.getAll()) {
            System.out.println(account);
        }

        System.out.println("\nAccounts with balance above Rs. 5000:");
        for (Account account : store.getAccountsAbove(5000)) {
            System.out.println(account);
        }
    }
}

class AccountTransactionTask implements Runnable {
    private final TransactionProcessor processor;
    private final String accountNumber;
    private final boolean deposit;
    private final long amount;

    public AccountTransactionTask(
            TransactionProcessor processor,
            String accountNumber,
            boolean deposit,
            long amount) {
        this.processor = processor;
        this.accountNumber = accountNumber;
        this.deposit = deposit;
```

```
        this.amount = amount;
    }

    @Override
    public void run() {
        Account account = processor.getAccount(accountNumber);

        if (account == null) {
            System.out.println("Account not found: " + accountNumber);
            return;
        }

        try {
            if (deposit) {
                account.deposit(amount);
                System.out.println("Deposited Rs. " + amount
                        + " into " + accountNumber);
            } else {
                account.withdraw(amount);
                System.out.println("Withdrew Rs. " + amount
                        + " from " + accountNumber);
            }
        } catch (InsufficientFundsException | InvalidAmountException e)
{
            System.out.println("Transaction failed: " + e.getMessage());
        }
    }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB12\MiniBank> New-Item -ItemType Directory -Force
PS D:\java\git java\JAVApracticals\LAB12\MiniBank> javac -d out (Get-ChildItem -Recurse
>> if ($LASTEXITCODE -eq 0) { java -cp out service.MiniBank }
Accounts sorted by balance:
Account Number: AC0001 | Owner: Bhavya | Balance: Rs. 5000 | Active: true
Account Number: AC0002 | Owner: Japan | Balance: Rs. 10000 | Active: true
Account Number: AC0003 | Owner: Shivam | Balance: Rs. 20000 | Active: true

Find account by number:
Account Number: AC0001 | Owner: Bhavya | Balance: Rs. 5000 | Active: true

Transaction history:
Bhavya opened account AC0001
Japan opened account AC0002
Shivam opened account AC0003
Deposited Rs. 500 into AC0001
Withdrew Rs. 1000 from AC0002

Balances after concurrent transactions:
Account Number: AC0001 | Owner: Bhavya | Balance: Rs. 5500 | Active: true
Account Number: AC0003 | Owner: Shivam | Balance: Rs. 20000 | Active: true
Account Number: AC0002 | Owner: Japan | Balance: Rs. 9000 | Active: true

Accounts with balance above Rs. 5000:
Account Number: AC0001 | Owner: Bhavya | Balance: Rs. 5500 | Active: true
Account Number: AC0003 | Owner: Shivam | Balance: Rs. 20000 | Active: true
Account Number: AC0002 | Owner: Japan | Balance: Rs. 9000 | Active: true
PS D:\java\git java\JAVApracticals\LAB12\MiniBank>
```

Practical 13

	service/Repository.java

CODE	package service;
	
	import java.lang.reflect.Field;
	import java.util.ArrayList;
	import java.util.HashMap;
	import model.annotation.Id;
	
	public class Repository<T> {
	  private final HashMap<String, T> data = new HashMap<>();
	
	  private String getId(T entity) {
	  if (entity == null) {
	  throw new IllegalArgumentException("Entity cannot be null.");
	  }
	
	  for (Class<?> type = entity.getClass();
	  type != null;
	  type = type.getSuperclass()) {
	
	  for (Field field : type.getDeclaredFields()) {

```
            if (field.isAnnotationPresent(Id.class)) {
                try {
                    field.setAccessible(true);
                    Object id = field.get(entity);

                    if (id == null) {
                        throw new IllegalArgumentException(
                                "The @Id field cannot be null."
                        );
                    }

                    return id.toString();
                } catch (IllegalAccessException e) {
                    throw new IllegalStateException(
                            "Could not read the @Id field.", e
                    );
                }
            }
        }
    }

    throw new IllegalArgumentException(
            "No field marked with @Id was found."
    );
}

public synchronized T save(T entity) {
    data.put(getId(entity), entity);
    return entity;
}

public synchronized T findById(String id) {
    return data.get(id);
}

public synchronized ArrayList<T> findAll() {
    return new ArrayList<>(data.values());
}

public synchronized boolean delete(String id) {
    return data.remove(id) != null;
}

@SafeVarargs
public final synchronized void saveAll(T... items) {
    for (T item : items) {
        save(item);
    }
}
```

```
    public static <N extends Number> double average(N[] values) {
        if (values == null || values.length == 0) {
            throw new IllegalArgumentException("Array cannot be
empty.");
        }

        double total = 0;
        for (N number : values) {
            total += number.doubleValue();
        }

        return total / values.length;
    }
}
```

model/Customer.java

CODE

```
package model;

import java.io.Serializable;
import model.annotation.Id;

public class Customer implements Cloneable, Serializable {
    private static final long serialVersionUID = 1L;

    private String name;
    private String email;
    private String mobile;

    @Id
    private final String customerId;

    private Address address;
    private static long customerCounter = 100;

    public Customer(String name, String email, String mobile) {
        this(name, email, mobile, null);
    }

    public Customer(
            String name,
            String email,
            String mobile,
            Address address) {
        this.name = name;
        this.email = email;
        this.mobile = mobile;
        this.address = address;
        this.customerId = generateCustomerId();
    }

    private static String generateCustomerId() {
```

```
        return "CUST" + (++customerCounter);
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getMobile() {
        return mobile;
    }

    public String getCustomerId() {
        return customerId;
    }

    public Address getAddress() {
        return address;
    }

    @Override
    public Customer clone() {
        try {
            return (Customer) super.clone();
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);
        }
    }

    public static class Address implements Serializable {
        private static final long serialVersionUID = 1L;

        private final String line;
        private final String city;
        private final String pincode;

        public Address(String line, String city, String pincode) {
            this.line = line;
            this.city = city;
            this.pincode = pincode;
        }

        public String getLine() {
            return line;
        }

        public String getCity() {
```

```
            return city;
        }

        public String getPincode() {
            return pincode;
        }

        @Override
        public String toString() {
            return line + ", " + city + " - " + pincode;
        }
    }
}
```

service/MiniBank.java

CODE

```
package service;

import model.Account;
import model.Customer;
import model.CurrentAccount;
import model.FixedDepositAccount;
import model.SavingsAccount;

public class MiniBank {
    public static void main(String[] args) {
        Repository<Account> accounts = new Repository<>();

        Account bhavyaAccount =
                new SavingsAccount("Bhavya", 5000, 1000);
        Account japanAccount =
                new CurrentAccount("Japan", 10000, 2000);
        Account shivamAccount =
                new FixedDepositAccount("Shivam", 0);

        accounts.saveAll(bhavyaAccount, japanAccount,
shivamAccount);

        System.out.println("Account found by ID:");
        Account foundAccount =
                accounts.findById(bhavyaAccount.getAccountNumber());
        System.out.println(foundAccount);

        System.out.println("\nAll accounts:");
        for (Account account : accounts.findAll()) {
            System.out.println(account);
        }

        Repository<Customer> customers = new Repository<>();

        Customer bhavya = new Customer(
                "Bhavya",
```

```
                "bhavya@example.com",
                "9876543210",
                new Customer.Address("Main Road", "Kheroli", "123456")
        );

        customers.save(bhavya);

        System.out.println("\nCustomer found by ID:");
        Customer foundCustomer =
                customers.findById(bhavya.getCustomerId());

        if (foundCustomer != null) {
            System.out.println("Customer ID: "
                    + foundCustomer.getCustomerId());
            System.out.println("Name: " + foundCustomer.getName());
            System.out.println("Email: " + foundCustomer.getEmail());
            System.out.println("Mobile: " + foundCustomer.getMobile());
            System.out.println("Address: " +
foundCustomer.getAddress());
        }

        System.out.println("\nAverage of the account balances:");
        System.out.println(
                Repository.average(new Long[] {5000L, 10000L, 0L})
        );
    }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB13\MiniBank> javac -d out (Get-ChildItem -Re
>> if ($LASTEXITCODE -eq 0) { java -cp out service.MiniBank }
Account found by ID:
Account Number: AC0001 | Owner: Bhavya | Balance: Rs. 5000 | Active: true

All accounts:
Account Number: AC0001 | Owner: Bhavya | Balance: Rs. 5000 | Active: true
Account Number: AC0003 | Owner: Shivam | Balance: Rs. 0 | Active: true
Account Number: AC0002 | Owner: Japan | Balance: Rs. 10000 | Active: true

Customer found by ID:
Customer ID: CUST101
Name: Bhavya
Email: bhavya@example.com
Mobile: 9876543210
Address: Main Road, Kheroli - 123456

Average of the account balances:
5000.0
PS D:\java\git java\JAVApracticals\LAB13\MiniBank>
```

Practical 14

	service/BankService.java

CODE	package service;
	
	import exception.AccountNotFoundException;
	import exception.BankException;
	import exception.InsufficientFundsException;
	import exception.InvalidAmountException;
	import java.io.IOException;
	import java.nio.file.Path;
	import java.util.ArrayList;
	import java.util.concurrent.CountDownLatch;
	import model.Account;
	import model.SavingsAccount;
	import util.ReportGenerator;
	import util.TransactionLog;
	import util.Validator;
	
	public class BankService {
	  private final Repository<Account> repository;
	  private final TransactionProcessor processor;
	  private final Path transactionLog;
	  private final Path reportFile;
	
	  public BankService(
	  Repository<Account> repository,
	  TransactionProcessor processor,
	  Path transactionLog,
	  Path reportFile) {
	  this.repository = repository;
	  this.processor = processor;
	  this.transactionLog = transactionLog;
	  this.reportFile = reportFile;
	  }

```
    public Account openAccount(
            String name,
            String email,
            String mobile,
            long openingBalance) throws InvalidAmountException {
        if (name == null || name.isBlank() || name.length() > 20) {
            throw new InvalidAmountException(
                    "Name must contain 1 to 20 characters."
            );
        }

        if (!Validator.isValidEmail(email)) {
            throw new InvalidAmountException("Invalid email address.");
        }

        if (!Validator.isValidMobile(mobile)) {
            throw new InvalidAmountException("Invalid mobile number.");
        }

        if (openingBalance < 0) {
            throw new InvalidAmountException(
                    "Opening balance cannot be negative."
            );
        }

        // Avoid reusing an account number after loading saved
accounts.
        Account account;
        do {
            account = new SavingsAccount(name, openingBalance, 0);
        } while (repository.get(account.getAccountNumber()) != null);

        registerAccount(account);
        return account;
    }

    public void registerAccount(Account account) {
        repository.save(account);
        processor.addAccount(account);
    }

    public void deposit(String accountNumber, long amount)
            throws BankException, IOException {
        Account account = findAccount(accountNumber);
        account.deposit(amount);
        TransactionLog.append(transactionLog, "DEPOSIT " + amount);
    }

    public void withdraw(String accountNumber, long amount)
```

```
        throws BankException, IOException {
      Account account = findAccount(accountNumber);
      account.withdraw(amount);
      TransactionLog.append(transactionLog, "WITHDRAW " +
amount);
   }

   public void transfer(String fromNumber, String toNumber, long
amount)
        throws BankException, IOException {
      Account from = findAccount(fromNumber);
      Account to = findAccount(toNumber);

      from.transfer(to, amount);
      TransactionLog.append(transactionLog, "TRANSFER " +
amount);
   }

   public String statement(String accountNumber)
        throws AccountNotFoundException {
      return findAccount(accountNumber).toString();
   }

   public ArrayList<Account> listAccounts() {
      return repository.findAll();
   }

   public String generateReport() throws IOException {
      return ReportGenerator.generate(
            transactionLog.getParent(),
            reportFile
      );
   }

   public void processDepositBatch(
         String accountNumber,
         int numberOfDeposits,
         long amount) throws AccountNotFoundException,
InterruptedException {
      if (numberOfDeposits <= 0 || amount <= 0) {
         throw new IllegalArgumentException(
               "Deposit count and amount must be positive."
         );
      }

      findAccount(accountNumber);

      CountDownLatch completed = new
CountDownLatch(numberOfDeposits);
```

```
        for (int i = 0; i < numberOfDeposits; i++) {
            processor.submit(() -> {
                try {
                    Account sharedAccount =
processor.getAccount(accountNumber);

                    if (sharedAccount != null) {
                        sharedAccount.deposit(amount);
                        TransactionLog.append(
                                transactionLog,
                                "DEPOSIT " + amount
                        );
                    }
                } catch (InvalidAmountException | IOException e) {
                    System.out.println("Batch transaction failed: "
                            + e.getMessage());
                } finally {
                    completed.countDown();
                }
            });
        }

        completed.await();
    }

    public void stopProcessor() throws InterruptedException {
        processor.stop();
    }

    private Account findAccount(String accountNumber)
            throws AccountNotFoundException {
        Account account = repository.findById(accountNumber);

        if (account == null) {
            throw new AccountNotFoundException(
                    "Account not found: " + accountNumber
            );
        }

        return account;
    }
}
```

service/MiniBank.java

CODE

```
package service;

import exception.BankException;
import java.nio.file.Files;
import java.nio.file.Path;
import model.Account;
import model.CurrentAccount;
```

```
import model.FixedDepositAccount;
import model.SavingsAccount;
import util.StatePersister;

public class MiniBank {
    public static void main(String[] args) throws Exception {
        Path dataFolder = Path.of("data");
        Path stateFile = dataFolder.resolve("accounts.dat");
        Path logFile =
dataFolder.resolve("logs").resolve("transactions.log");
        Path reportFile = dataFolder.resolve("reports").resolve("eod-
report.txt");

        Files.createDirectories(dataFolder);

        Repository<Account> repository = new Repository<>();
        TransactionProcessor processor = new TransactionProcessor();
        BankService bank = new BankService(
                repository,
                processor,
                logFile,
                reportFile
        );

        if (Files.exists(stateFile)) {
            for (Account account : StatePersister.load(stateFile)) {
                bank.registerAccount(account);
            }
            System.out.println("Saved accounts loaded.");
        } else {
            // Your account data from the earlier MiniBank practicals.
            bank.registerAccount(new SavingsAccount("Bhavya", 5000,
1000));
            bank.registerAccount(new CurrentAccount("Japan", 10000,
2000));
            bank.registerAccount(new FixedDepositAccount("Shivam",
0));
        }

        boolean running = true;

        try (var scanner = new java.util.Scanner(System.in)) {
            while (running) {
                System.out.println("\n====== MiniBank ======");
                System.out.println("1. Open Account");
                System.out.println("2. Deposit");
                System.out.println("3. Withdraw");
                System.out.println("4. Transfer");
                System.out.println("5. Statement");
                System.out.println("6. List Accounts");
```

```
System.out.println("7. Process Deposit Batch");
System.out.println("8. End-of-Day Report");
System.out.println("9. Exit");
System.out.print("Enter choice: ");

String choice = scanner.nextLine().trim();

try {
   switch (choice) {
      case "1" -> {
         System.out.print("Enter name: ");
         String name = scanner.nextLine();

         System.out.print("Enter email: ");
         String email = scanner.nextLine();

         System.out.print("Enter mobile: ");
         String mobile = scanner.nextLine();

         System.out.print("Enter opening balance: ");
         long openingBalance =
               Long.parseLong(scanner.nextLine());

         Account account = bank.openAccount(
               name, email, mobile, openingBalance
         );
         System.out.println("Account opened: " + account);
      }

      case "2" -> {
         showAccounts(bank);
         System.out.print("Enter account number: ");
         String accountNumber = scanner.nextLine();

         System.out.print("Enter deposit amount: ");
         long amount = Long.parseLong(scanner.nextLine());

         bank.deposit(accountNumber, amount);
         System.out.println("Deposit successful.");
      }

      case "3" -> {
         showAccounts(bank);
         System.out.print("Enter account number: ");
         String accountNumber = scanner.nextLine();

         System.out.print("Enter withdrawal amount: ");
         long amount = Long.parseLong(scanner.nextLine());

         bank.withdraw(accountNumber, amount);
```

```
                    System.out.println("Withdrawal successful.");
                }

                case "4" -> {
                    showAccounts(bank);
                    System.out.print("Transfer from account: ");
                    String from = scanner.nextLine();

                    System.out.print("Transfer to account: ");
                    String to = scanner.nextLine();

                    System.out.print("Enter transfer amount: ");
                    long amount = Long.parseLong(scanner.nextLine());

                    bank.transfer(from, to, amount);
                }

                case "5" -> {
                    System.out.print("Enter account number: ");
                    String accountNumber = scanner.nextLine();

System.out.println(bank.statement(accountNumber));
                }

                case "6" -> showAccounts(bank);

                case "7" -> {
                    showAccounts(bank);
                    System.out.print("Enter account number: ");
                    String accountNumber = scanner.nextLine();

                    System.out.print("Number of deposits: ");
                    int count = Integer.parseInt(scanner.nextLine());

                    System.out.print("Amount per deposit: ");
                    long amount = Long.parseLong(scanner.nextLine());

                    bank.processDepositBatch(
                            accountNumber, count, amount
                    );
                    System.out.println("Deposit batch completed.");
                }

                case "8" -> System.out.println(bank.generateReport());

                case "9" -> running = false;

                default -> System.out.println(
                        "Invalid choice. Enter a number from 1 to 9."
                );
```

```
                }
            } catch (BankException e) {
                System.out.println("Banking error: " + e.getMessage());
            } catch (NumberFormatException e) {
                System.out.println("Please enter a valid whole number.");
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                System.out.println("The operation was interrupted.");
            } catch (Exception e) {
                System.out.println("Operation failed: " +
e.getMessage());
            }
        }
    } finally {
        // Finish queued transactions before saving application state.
        bank.stopProcessor();
        StatePersister.save(
                bank.listAccounts().toArray(new Account[0]),
                stateFile
        );
        System.out.println("Accounts saved. Goodbye!");
    }
}

private static void showAccounts(BankService bank) {
    System.out.println("\nAccounts:");
    for (Account account : bank.listAccounts()) {
        System.out.println(account);
    }
}
}
```

test/service/BankServiceTest.java

CODE

```
package service;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import exception.InsufficientFundsException;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import model.Account;
import model.SavingsAccount;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;
```

```
class BankServiceTest {
    @TempDir
    Path tempDirectory;

    private TransactionProcessor processor;
    private BankService bank;

    @BeforeEach
    void setUp() {
        processor = new TransactionProcessor();

        bank = new BankService(
                new Repository<>(),
                processor,
                tempDirectory.resolve("logs").resolve("test.log"),
                tempDirectory.resolve("report.txt")
        );
    }

    @AfterEach
    void tearDown() throws InterruptedException {
        bank.stopProcessor();
    }

    @Test
    void depositIncreasesBalance() throws Exception {
        Account account = new SavingsAccount("Test User", 1000, 0);
        bank.registerAccount(account);

        bank.deposit(account.getAccountNumber(), 500);

        assertEquals(1500, account.getBalance());
    }

    @Test
    void withdrawalOverBalanceThrowsInsufficientFundsException()
            throws Exception {
        Account account = new SavingsAccount("Test User", 1000, 0);
        bank.registerAccount(account);

        assertThrows(
                InsufficientFundsException.class,
                () -> bank.withdraw(account.getAccountNumber(), 1500)
        );
    }

    @Test
    void concurrentDepositsKeepCorrectBalance() throws Exception {
        Account account = new SavingsAccount("Test User", 0, 0);
        int threadCount = 8;
```

```
        int depositsPerThread = 1000;

        ExecutorService executor =
                Executors.newFixedThreadPool(threadCount);
        ArrayList<Future<?>> futures = new ArrayList<>();

        try {
            for (int thread = 0; thread < threadCount; thread++) {
                futures.add(executor.submit(() -> {
                    for (int i = 0; i < depositsPerThread; i++) {
                        try {
                            account.deposit(1);
                        } catch (Exception e) {
                            throw new RuntimeException(e);
                        }
                    }
                }));
            }

            for (Future<?> future : futures) {
                future.get();
            }
        } finally {
            executor.shutdown();
        }

        assertEquals((long) threadCount * depositsPerThread,
                account.getBalance());
    }
}
```

README.md

```
# MiniBank

MiniBank is a Java console banking application. It supports account creation,
deposits, withdrawals, transfers, statements, account listing, concurrent
deposit batches, saved account state, and end-of-day transaction reports.

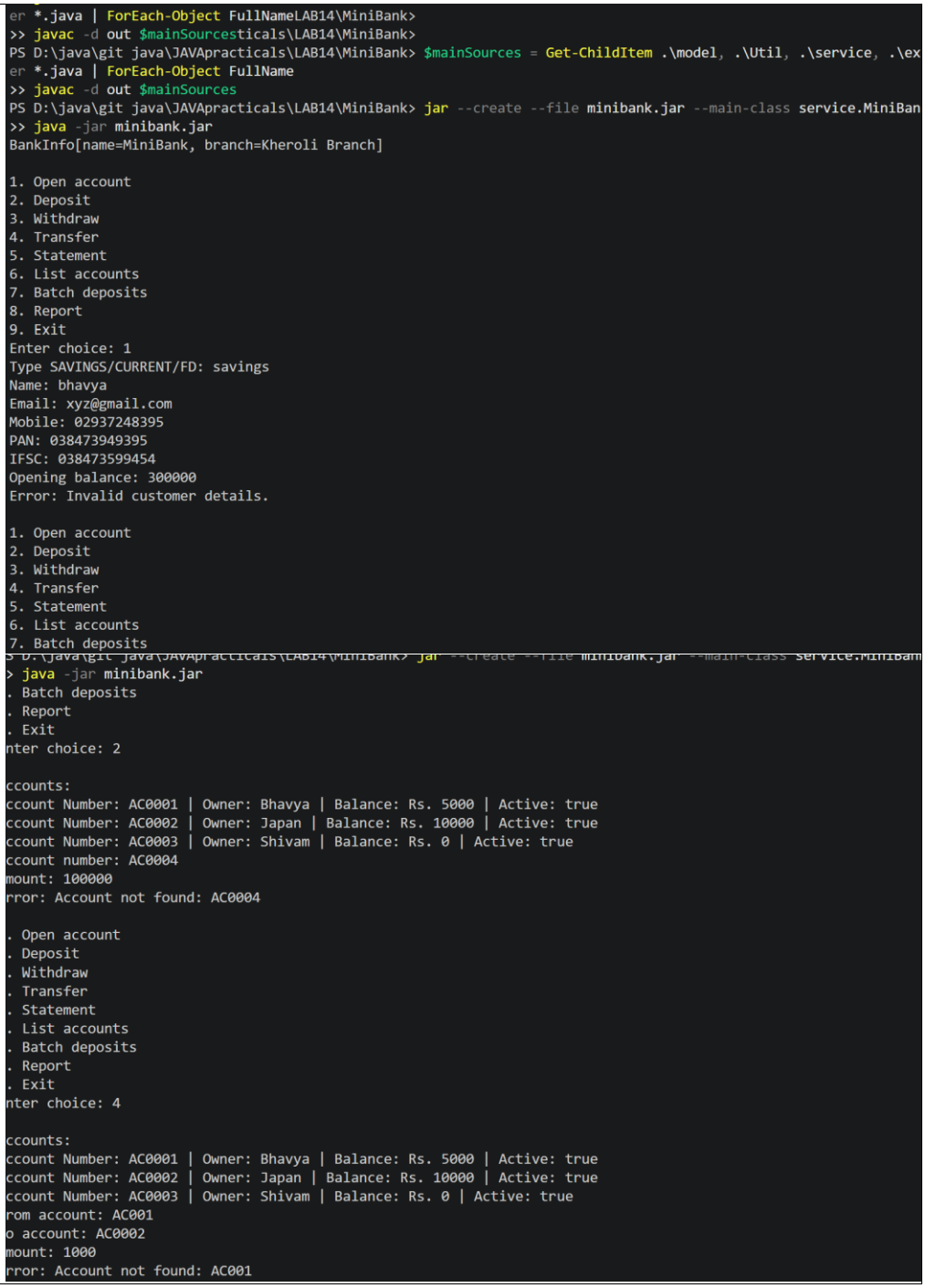
## Requirements

- JDK 17 or later
- JUnit 5 standalone JAR for running tests

## Compile the application

Run these PowerShell commands from the project root:

```powershell
$mainSources = Get-ChildItem .\model, .\util, .\service, .\exception `
```
```

	-Recurse -Filter *.java | ForEach-Object FullName javac -d out $mainSources
OUTPUT	
	

```
er *.java | ForEach-Object FullNameLAB14\MiniBank>
>> javac -d out $mainSourcesticals\LAB14\MiniBank>
PS D:\java\git java\JAVApracticals\LAB14\MiniBank> $mainSources = Get-ChildItem .\model, .\Util, .\service, .\ex
er *.java | ForEach-Object FullName
>> javac -d out $mainSources
PS D:\java\git java\JAVApracticals\LAB14\MiniBank> jar --create --file minibank.jar --main-class service.MiniBan
>> java -jar minibank.jar
BankInfo[name=MiniBank, branch=Kheroli Branch]

1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List accounts
7. Batch deposits
8. Report
9. Exit
Enter choice: 1
Type SAVINGS/CURRENT/FD: savings
Name: bhavya
Email: xyz@gmail.com
Mobile: 02937248395
PAN: 038473949395
IFSC: 038473599454
Opening balance: 300000
Error: Invalid customer details.

1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List accounts
7. Batch deposits
> java -jar minibank.jar
. Batch deposits
. Report
. Exit
nter choice: 2

ccounts:
ccount Number: AC0001 | Owner: Bhavya | Balance: Rs. 5000 | Active: true
ccount Number: AC0002 | Owner: Japan | Balance: Rs. 10000 | Active: true
ccount Number: AC0003 | Owner: Shivam | Balance: Rs. 0 | Active: true
ccount number: AC0004
mount: 100000
rror: Account not found: AC0004

. Open account
. Deposit
. Withdraw
. Transfer
. Statement
. List accounts
. Batch deposits
. Report
. Exit
nter choice: 4

ccounts:
ccount Number: AC0001 | Owner: Bhavya | Balance: Rs. 5000 | Active: true
ccount Number: AC0002 | Owner: Japan | Balance: Rs. 10000 | Active: true
ccount Number: AC0003 | Owner: Shivam | Balance: Rs. 0 | Active: true
rom account: AC001
o account: AC0002
mount: 1000
rror: Account not found: AC001
```

```
PS D:\java\git java\JAVApracticals\LAB14\MiniBank>
>> java -jar minibank.jar
Amount: 1000
Error: Account not found: AC001

1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List accounts
7. Batch deposits
8. Report
9. Exit
Enter choice: 5
Account number: AC0001
------ MiniBank Statement ------
Account number: AC0001
Owner: Bhavya
Balance: Rs. 5000
Active: true
--------------------------------
Transactions


1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List accounts
7. Batch deposits
8. Report
9. Exit
```

```
Accounts:
Account Number: AC0001 | Owner: Bhavya | Balance: Rs. 5000 | Active
Account Number: AC0002 | Owner: Japan | Balance: Rs. 10000 | Active
Account Number: AC0003 | Owner: Shivam | Balance: Rs. 0 | Active: t

1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List accounts
7. Batch deposits
8. Report
9. Exit
Enter choice: 7

Accounts:
Account Number: AC0001 | Owner: Bhavya | Balance: Rs. 5000 | Active
Account Number: AC0002 | Owner: Japan | Balance: Rs. 10000 | Active
Account Number: AC0003 | Owner: Shivam | Balance: Rs. 0 | Active: t
Account number: AC0004
Number of deposits: 20000
Amount per deposit: 10000
Error: Account not found: AC0004

1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List accounts
```

```
>> java -jar minibank.jar
Amount per deposit: 10000
Error: Account not found: AC0004

1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List accounts
7. Batch deposits
8. Report
9. Exit
Enter choice: 8
MiniBank End-of-Day Report

Total deposits: Rs. 0
Total withdrawals: Rs. 0


1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List accounts
7. Batch deposits
8. Report
9. Exit
Enter choice: 9
Accounts saved. Goodbye!
PS D:\java\git java\JAVApracticals\LAB14\MiniBank>
```